

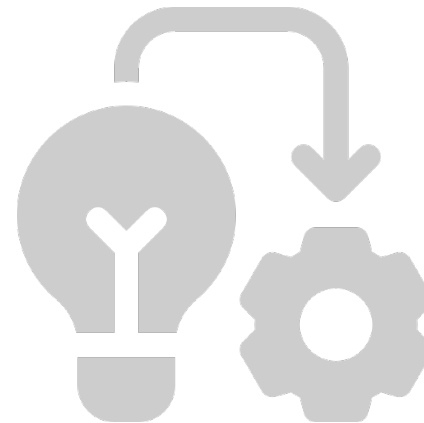
Regelungstechnik

Teil 7: Implementierung, Simulation und Anwendung

Prof. Dr.-Ing. Thomas Schulte

Fachbereich Elektrotechnik und Technische Informatik
Regelungstechnik und Simulation

Implementierung



Implementierung

Realisierung von Reglern

Regler wurden in der Vergangenheit analog, d. h. über Anlogschaltungstechnik (z. B. Operationsverstärkerschaltungen) oder auch mechanisch (bzw. pneumatisch und hydraulisch), realisiert. Heute werden analoge Regler fast nur noch für einfache aber sehr dynamische Regelkreise verwendet, z. B. die Strom- und Spannungsregelung bei einfachen Schaltnetzteilen. Ansonsten werden inzwischen für fast alle Regelungsaufgaben in allen Bereichen (Fahrzeugtechnik, Luftfahrt, Industrieanlagen, Haustechnik, Haushaltsgeräte, usw.) **Digitale Regler** eingesetzt.

Digitale Regler werden als Algorithmen auf Digitalrechnern (μC , μP , SPS, IPC, FPGA, ...) implementiert. Die Koppelung mit dem analogen Prozess (Regelstrecke) erfolgt über Ein- und Ausgabe-Umsetzer (Analog-Digital-Umsetzer - ADU und Digital-Analog-Umsetzer - DAU).

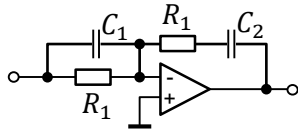
Vorteile der digitalen Regelungstechnik:

- Vermeidung der übliche Probleme der Analogtechnik: Offset-Drift, Störungen
- Flexibilität: Der Regler ist „nur“ Software → schnelle Anpassungen, Parametrierungen
- Einfachere Realisierung komplexer Regler
- Integration zusätzlicher Funktionen: Überwachung, Notlaufstrategie, Kommunikation
- Anbindungen moderner Kommunikation per Datenbus

Implementierung

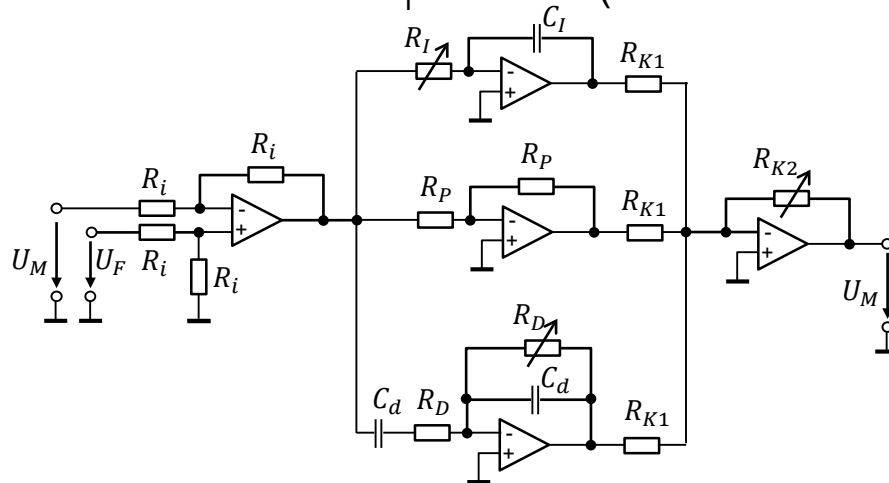
Analoge (elektronische) Reglerimplementierung:

Kompakte Form eines PID- T_1 -Gliedes:



$$K_p = \frac{R_2}{R_1} + \frac{C_1}{C_2}, \quad K_i = \frac{1}{R_1 C_2}, \quad K_d = R_2 C_1$$

Summenform eines PID- T_1 -Gliedes: (Bessere Einstellbarkeit der Reglerparameter)



$$K_p = \frac{R_{K2}}{R_{K1}}$$

$$T_N = R_I C_I$$

$$T_V = R_D C_D$$

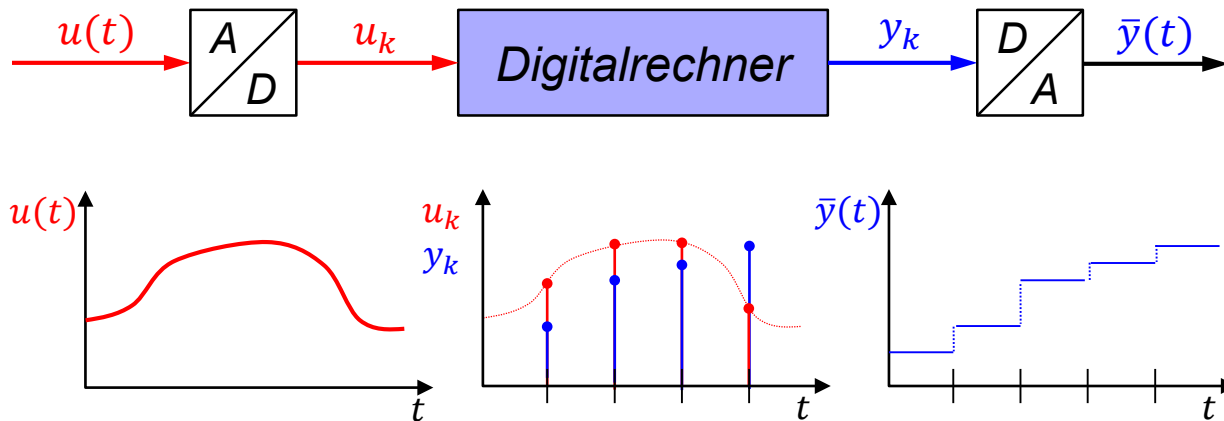
Eigenschaften analoger Regler:

- Relativ unflexibel, ARW-Maßnahme ist aufwendig
- Bauelemente ändern Parameter (z.B. bei Erwärmung) → Offset- und Verstärkungsdrift
- Hohe Dynamik → bei schnellen Strecken teilw. nicht durch digitale Regler abbildbar.

Implementierung

Abtastende Arbeitsweise

Der digitale Regler lässt sich gerätetechnisch (von außen) kaum von anderen Systemen der digitalen Signalverarbeitung (z. B. digitale Filterung von Audiosignalen) unterscheiden:

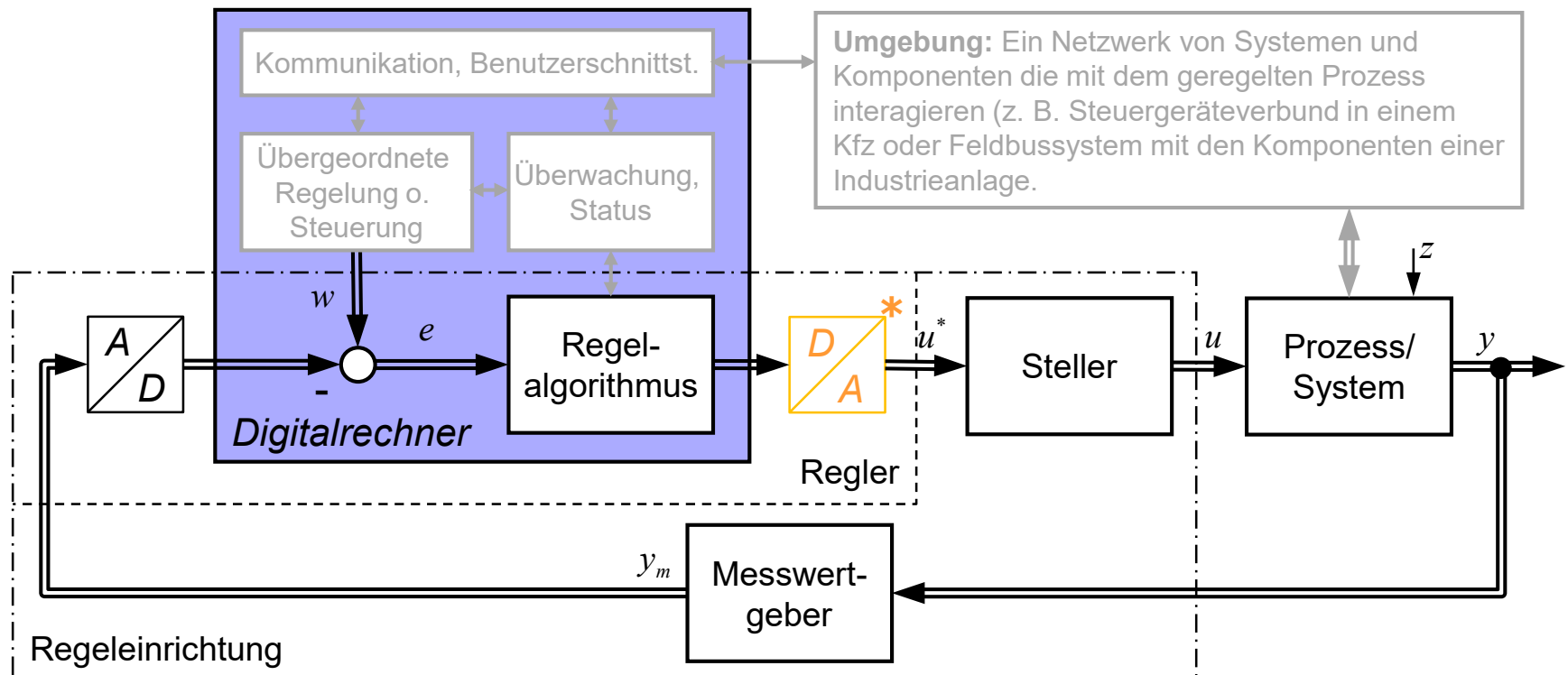


Entscheidend gegenüber den analogen Lösungen ist die abtastende Wirkungsweise und die Amplituden-Quantisierung (gilt auch für andere Arten der Signalverarbeitung). Dadurch geht der Verlauf der zeit- und wertkontinuierliche Prozesssignale in wert- und zeitdiskrete Größen auf dem digitalen Rechensystem über. Digitale Regelungen werden deshalb auch als **diskrete Regelungen** oder **Abtastregelungen** bezeichnet.

Implementierung

Digitale Regelung mit Zusatzfunktionen (Überblick)

Die Implementierung eines Reglers auf einem digitalen Rechensystem lässt die einfache Realisierung von Zusatzfunktionen zu, wobei dem Regelalgorithmus aufgrund der Dynamik die höchste Priorität gegeben werden muss.



* Der Regler wirkt i.d.R. nicht über einen konventionellen DAU, sondern über einen äquivalenten Prozess, z. B. eine PWM modulierte Leistungsstufe (s. Kap. 6).

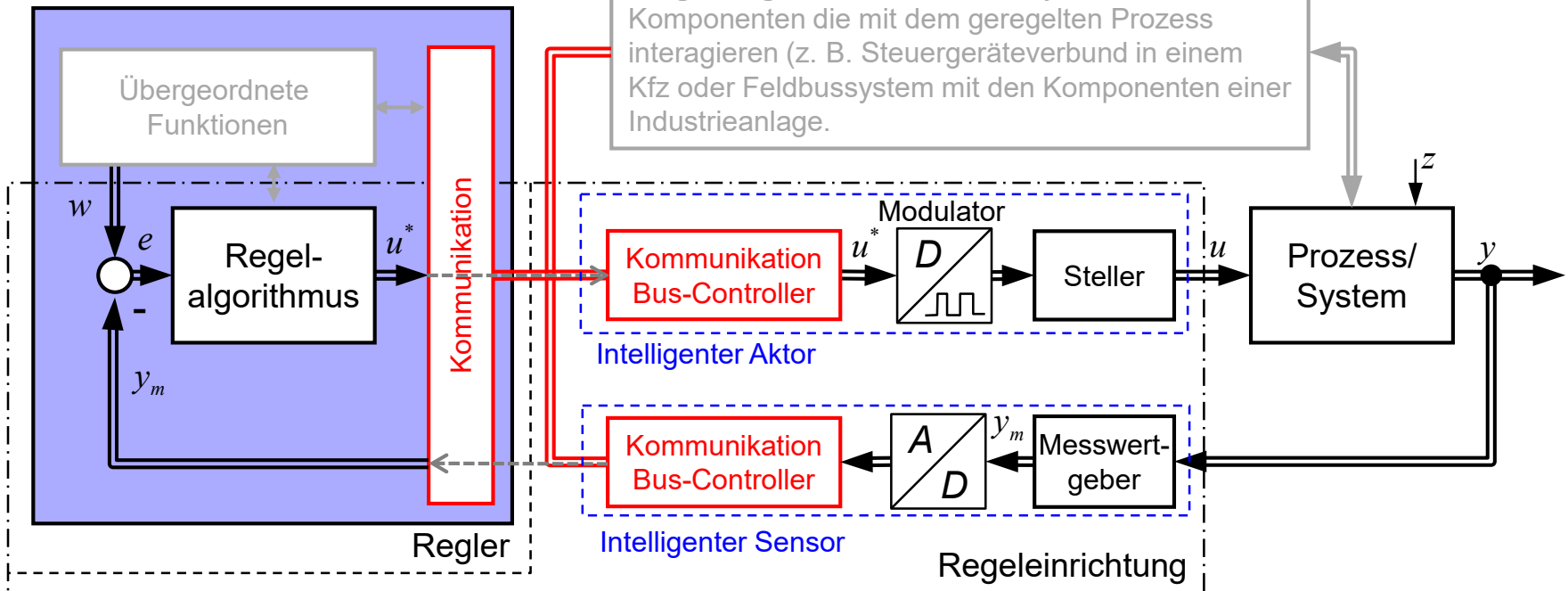
Implementierung

Digitale Regelung mit Netzwerkanbindung (Überblick)

Die Anbindung eines Reglers an seine Aktoren (Stellglied) und Sensoren (Messglied) über einen Datenbus oder ein Datennetzwerk erfordert üblicherweise auch die Echtzeitfähigkeit der Kommunikation, um definierte Latenzen und Abtastzeiten sicherzustellen.

SPS, IPC oder ECU (Kfz, Avionik)

Umgebung: Ein Netzwerk von Systemen und Komponenten die mit dem geregelten Prozess interagieren (z. B. Steuergeräteverbund in einem Kfz oder Feldbussystem mit den Komponenten einer Industrieanlage).



==== Datenbus/Datennetzwerk (CAN, EtherCat, Flexray,...), idealerweise echtzeitfähig

Implementierung

Anforderungen an digitale Regler

Die „digitale Realisierung“ eines Regler bedeutet nicht, dass die bisher in Bezug auf kontinuierliche (analoge) Regler besprochene Theorie obsolet ist:

- Die Regelstrecke ist weiterhin ein reales kontinuierliches System.
- Strukturen wie z. B. Kaskadenregelung, Störgrößenaufschaltung usw. können direkt auf digitale Regler übertragen werden.
- Reglertypen (PI, PID, ...) können als digitale Regler implementiert werden.
- Entwurfsverfahren und Stabilitätsbetrachtungen können mit Anpassungen auf die digitale Regelungstechnik übertragen werden.

Es existiert eine eigenständige Theorie der **diskreten Systeme** (und Signale), die auch in anderen Bereichen der digitalen Datenverarbeitung Verwendung findet. Diese soll hier allerdings nicht behandelt werden (s. RT2)

Nichtsdestotrotz sind aufgrund der zeitdiskreten, abtastende Arbeitsweise digitaler Regler folgende Fragen zu klären:

- Wie können digitale Regler als Algorithmus äquivalent zu kontinuierlichen Regler implementiert werden?
- Welchen dynamischen Einfluss hat die abtastende Arbeitsweise digitaler Regler?
- Welche Anforderungen bestehen hinsichtlich der Abtastfrequenz und der Echtzeitfähigkeit der digitalen Signalverarbeitung?

Implementierung

Diskrete Signalbeschreibung:

Digitale Systeme können nur zeitdiskrete Signale verarbeiten, bei denen zeitlichen Stützstellen $t_0, t_1, t_2, \dots$ zugeordnete Werte $x_0 = x(t_0), x_1 = x(t_1), x_2 = x(t_2), \dots$ eines Signals zur weiteren Verarbeitung, Analyse oder Darstellung abgelegt werden. Unabhängig von den kontinuierlichen Signal- und Systembeschreibungen können wir diese Werte im Sinne einer eigenständigen diskreten Signal- und Systemtheorie mathematisch als Zahlenfolgen auffassen:

$$(t_k) = (t_0, t_1, t_2, \dots) \quad \text{und} \quad (x_k) = (x_0, x_1, x_2, \dots)$$

Bei Systemen zur Echtzeitverarbeitung von Signalen (digitale Regler, HIL-Simulation, digitale Filter, Audio- und Videodatenübertragung) liegen fast ausschließlich konstante Abtastfrequenzen f vor, sodass mit

$$(t_k) = (0, T, 2T, 3T, \dots) = kT \quad \text{mit} \quad T = \frac{1}{f}$$

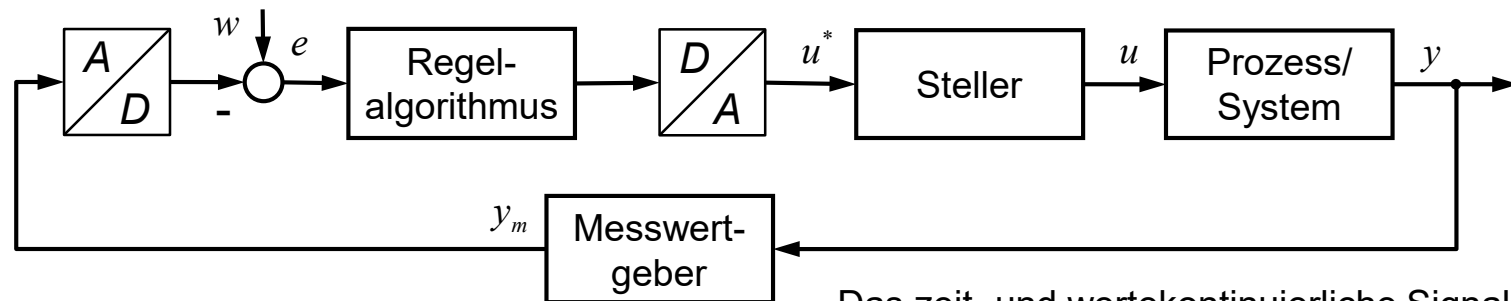
der Abstand T zwischen zwei Stützstellen ebenfalls konstant ist.

Da es sich bei den Werten $x_k = x(t_k)$ üblicherweise tatsächlich um die einem kontinuierlichen Signal zu den Zeitpunkten t_k zugeordneten Werte handelt, die z. B. durch „Abtastung“ (Messung per ADU) dieses Signals zustande kommen, wird T als „Abtastzeit“ oder „Abtastschrittweite“ bezeichnet. Bei konstantem T ist das diskrete Signal damit durch die Folge (x_k) vollständig charakterisiert und beschrieben.

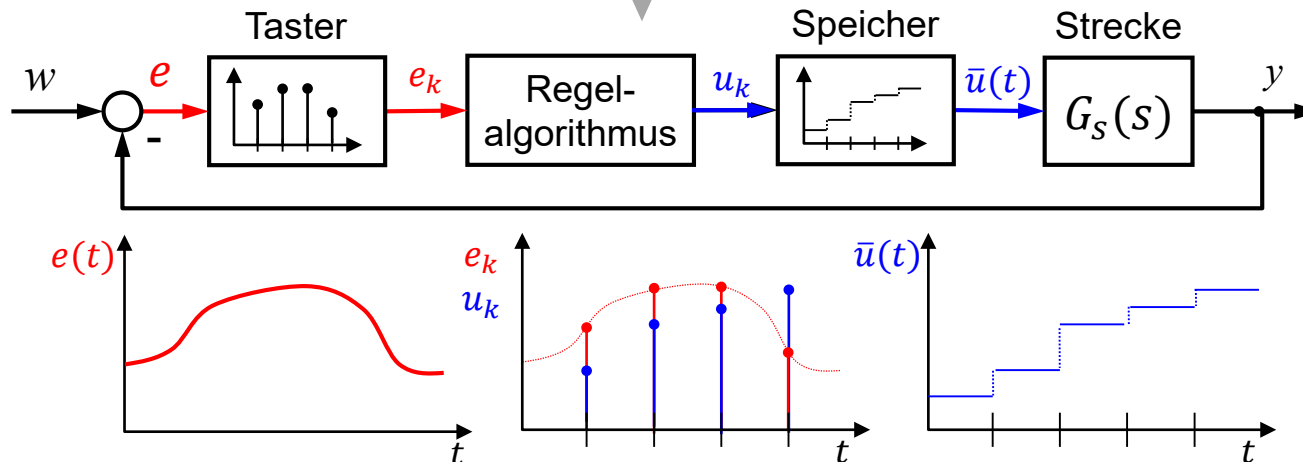
Implementierung

Abtastung

Der digitale Regelkreis wird weiter abstrahiert:



Das zeit- und wertekontinuierliche Signal $e(t)$ wird durch den Abtastvorgang in ein zeit- und wertediskretes Signal umgesetzt.



Implementierung

Nyquist-Shannon-Abtasttheorem

Das **Nyquist-Shannon-Abtasttheorem** (teilw. nur **Shannonsches Abtasttheorem**) wurde von diversen Wissenschaftlern unabhängig voneinander und in mathematisch unterschiedlicher Weise formuliert, wobei die Kernaussage identisch ist:

Ein Signal $x(t)$ muss mindestens mit der doppelten Frequenz seiner höchsten Frequenzanteile abgetastet werden, da ansonsten Fremdfrequenz durch Überlappung von Spektralanteilen (auch „Schatten“ genannt) entstehen. Umgekehrt, kann ein entsprechend dieser Regel bandbegrenzttes Signal eindeutig aus seinen Abtastwerten x_k rekonstruiert werden, was allerdings eine unendliche Beobachtungszeit ($k = -\infty \dots \infty$) erfordert.

Hinweise:

1. Da sich Band- und Zeitbegrenztheit der Theorie nach ausschließen und jedes reale Signal naturgemäß zeitbegrenzt ist, also nicht bandbegrenzt sein kann, ist die Einhaltung des Nyquist-Shannon-Abtasttheorem eigentlich nicht möglich.
2. In der Praxis werden höhere Frequenzanteile abzutastender Signale durch **Anti-Aliasing-Filter** soweit unterdrückt, dass der Aliasing-Effekt unterhalb der Messgenauigkeit (Quantisierung, Toleranz der Messwertaufnehmer) liegt.
3. Aliasing-Effekte treten auch bei der Signalübergabe zwischen unterschiedlich schnell abtastenden diskreten Systemen auf (höherfrequent → niederfrequent).

Implementierung

Quasikontinuierlicher Reglerentwurf

Beim quasikontinuierlichen Reglerentwurf werden die bekannten Verfahren des kontinuierlichen Reglerentwurfs angewendet, in der Annahme das Verhalten des kontinuierlichen Reglers durch einen diskreten Regler (Regelalgorithmus) in näherungsweise hinreichender Form nachbilden zu können. Der diskrete (digitale) Regler wird also als quasikontinuierlich betrachtet → “quasikontinuierlichen Regelungen”.

Vorteile:

- Vorhandene Entwicklungswerkzeuge können genutzt werden.
- Der Entwurf „im Kontinuierlichen“ entspricht dem Charakter der kontinuierlichen Strecke und die bekannten Methoden (z. B. Frequenzbereich) ermöglichen eine bessere Interpretation als bei diskreter Darstellung.

Nachteile:

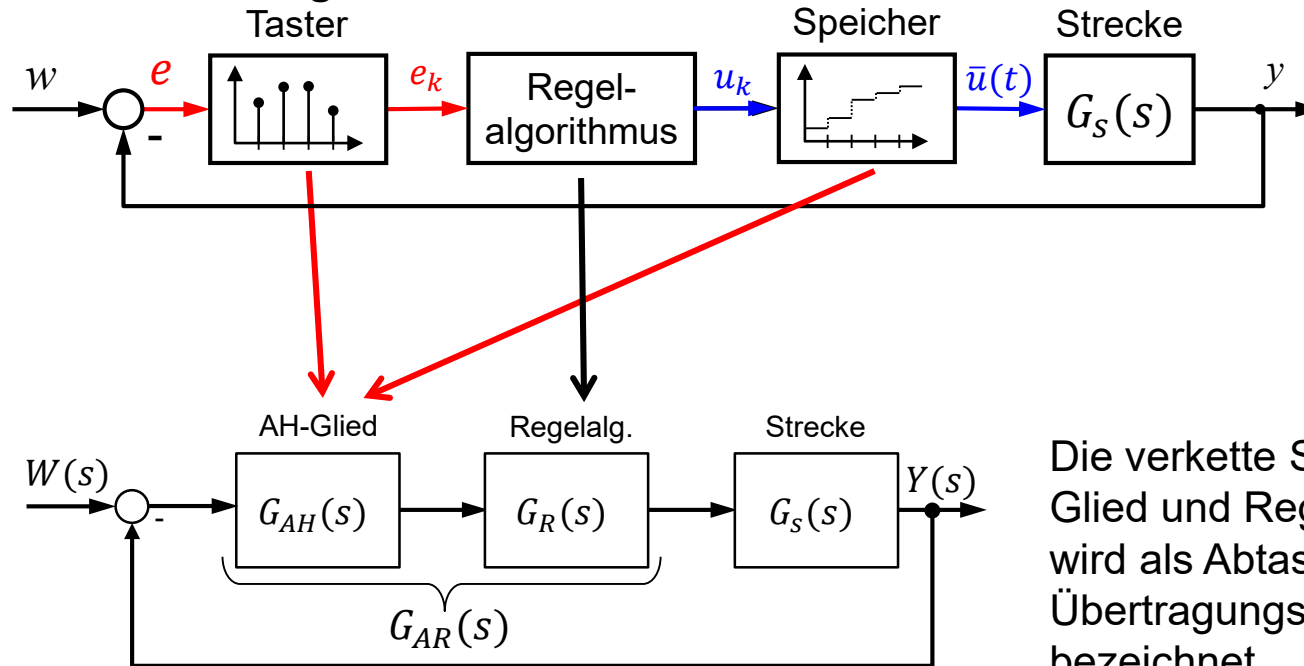
- Die quasikontinuierliche Betrachtung bedingte eine ausreichend hohe Abtastfrequenz.
- Weniger leistungsfähig als angepasste Entwurfsmethoden „im Diskreten“.

Anmerkung:

Der quasikontinuierliche Reglerentwurf ist die in der industriellen Praxis wahrscheinlich am häufigsten angewandte Methode.

Implementierung

Approximation des Regelkreises im s-Bereich:



Die verkettete Struktur von AH-Glied und Regelalgorithmus wird als Abtastregler mit der Übertragungsfunktion $G_{AR}(s)$ bezeichnet.

Regelalgorithmus

Die Reglerübertragungsfunktion $G_R(s)$ muss durch eine geeignete Differenzengleichung als digital implementierbare Approximation beschrieben werden:

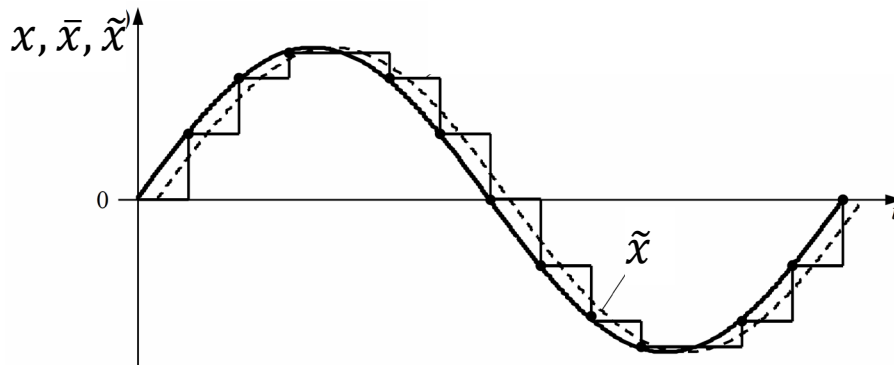
$$\begin{aligned} &\alpha_n \cdot u_k + \alpha_{n-1} \cdot u_{k-1} + \dots + \alpha_1 \cdot u_{k-n+1} + \alpha_0 \cdot u_{k-n} \\ &= \beta_n \cdot e_k + \beta_{n-1} \cdot e_{k-1} + \dots + \beta_1 \cdot e_{k-n+1} + \beta_0 \cdot e_{k-n} \end{aligned}$$

Implementierung

Approximation des Abtast-Halteglieds:

Wird der (geringe) Amplitudenabfall vernachlässigt, so kann das Abtast-Halteglied im Frequenzbereich näherungsweise durch das Totzeitglied approximiert werden:

$$G_{AH}(j\omega) \approx e^{-j\frac{T}{2}\omega} \quad \text{Totzeitapproximation mit } T_t = \frac{T}{2}$$



$$\tilde{x}(t) \quad \longleftrightarrow \quad \tilde{X}(s) = e^{-s \cdot \frac{T}{2}} \cdot X(s)$$

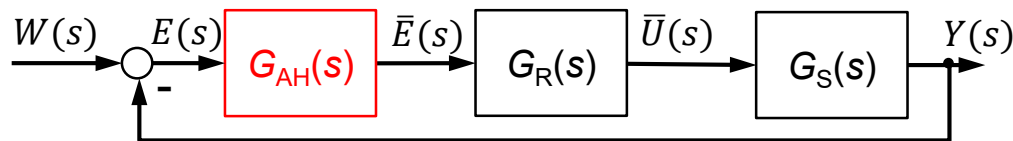
Als weitere Vereinfachung kann das Totzeitglied (wie häufig) durch einen Verzögerungsglied 1. Ordnung (P-T₁-Glied) approximiert werden (vgl. Übung 7):

$$G_{AH}(j\omega) \approx \left. \frac{1}{1 + s \cdot \frac{T}{2}} \right|_{s=j\omega} = \frac{1}{1 + j\omega \cdot \frac{T}{2}} \quad \text{P-T}_1\text{-Näherung mit } T_1 = \frac{T}{2}$$

Implementierung

Vorgehensweise beim quasikontinuierlicher Reglerentwurf:

1. Die Strecke wird wie üblich als kontinuierliche Strecke modelliert.
2. Das AH-Glied wird in geeigneter Weise berücksichtigt:



- Totzeitapproximation $G_{AH}(s) \approx e^{-\frac{T}{2}s}$ (s.o.): In der Regel mit weiteren kleinen Streckenzeitkonstanten als Anteil $T/2$ der Summenzeitkonstanten.
 - Da der diskrete Regler ein Algorithmus auf einem μC oder μP ist, kommt es durch die Berechnung (und ggf. Kommunikation) zu einer verzögerten Ausgabe der Stellgröße. Die zusätzlich zu berücksichtigende **Rechentotzeit** T_D ergibt mit $T/2$ üblicherweise eine Gesamttotzeit von $\geq T$.
3. Entwurf des Reglers (Struktur und Parameter) mit einer geeigneten Methode im Kontinuierlichen:

$$G_o(s) = G_S(s) \cdot G_R(s) \cdot G_{AH}(s) \cdot e^{-T_D s}$$

4. Approximation des kontinuierlichen Reglers durch einen diskreten Regler (diskretes System).

→ Der letzte Schritt wird im Folgenden noch etwas genauer betrachtet!

Implementierung

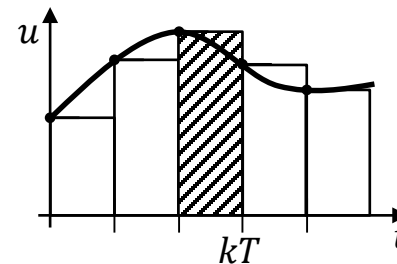
Approximation kont. Übertragungsglieder als Filter bzw. Regelalgorithmus

Die näherungsweise Überführung kontinuierlicher Systeme in eine digital rechenbare Differenzengleichung gelingt bei ausreichend hohen Abtastfrequenzen durch relativ einfache Approximationen der zeitlichen Integration (I-Glied):

1. Rechteckregel (Euler) vorwärts

Näherungsweise Integration über Rechteckblöcke:

$$y(t) = y(t_0) + \int_{t_0}^t u(t) dt \rightarrow y_k = y_{k-1} + T u_{k-1}$$

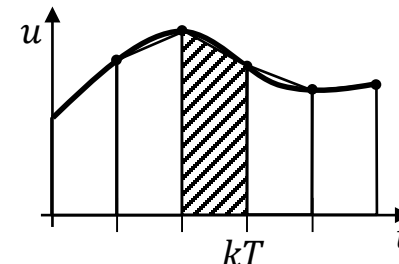


Diese Methode ist für die Integration allgemein als (vorwärts) Euler-Verfahren bekannt.

2. Trapezregel (Tustin)

Näherungsweise Integration über Trapezblöcke:

$$y(t) = y(t_0) + \int_{t_0}^t u(t) dt \rightarrow y_k = y_{k-1} + \frac{T}{2} \cdot (u_k + u_{k-1})$$



Üblicherweise wird diese „Trapezregel“ beim quasikontinuierlichen Reglerentwurf zur Überführung kontinuierlich formulierter Regler in einen Regelalgorithmus genutzt.

Implementierung

Approximation des PI-Reglers:

Summenform:

$$u_{I,k} = u_{I,k-1} + K_I \cdot \frac{T}{2} \cdot (e_k + e_{k-1})$$

$$u_k = u_{P,k} + u_{I,k} = K_p \cdot e_k + u_{I,k}$$

oder

$$u_{I,k} = u_{I,k-1} + K_p \cdot \frac{T}{2T_n} \cdot (e_k + e_{k-1})$$

$$u_k = u_{P,k} + u_{I,k} = K_p \cdot e_k + u_{I,k}$$

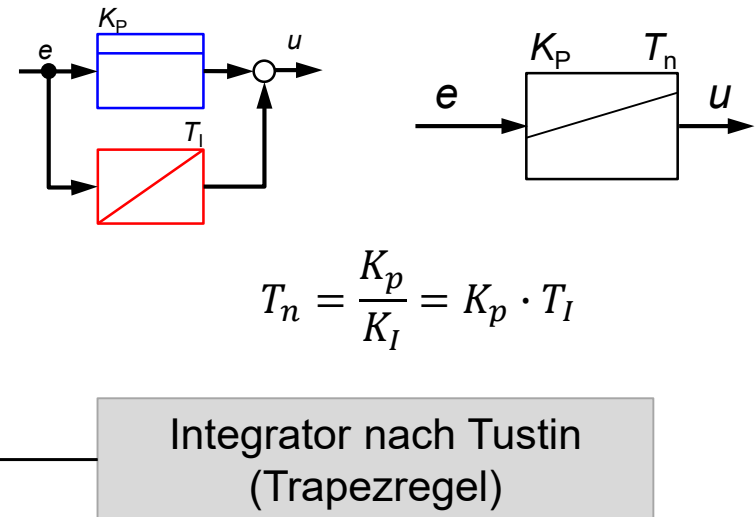
Differenzengleichung in Standardform:

$$u_k = K_p \cdot e_k + u_{I,k-1} + K_I \cdot \frac{T}{2} \cdot (e_k + e_{k-1})$$

$$u_k = K_p \cdot e_k + u_{I,k} \Leftrightarrow u_{k-1} = K_p \cdot e_{k-1} + u_{I,k-1} \Leftrightarrow u_{I,k-1} = u_{k-1} - K_p \cdot e_{k-1}$$

$$= K_p e_k + u_{k-1} - K_p e_{k-1} + K_I \frac{T}{2} (e_k + e_{k-1}) = u_{k-1} + \left(K_I \frac{T}{2} + K_p \right) e_k + \left(K_I \frac{T}{2} - K_p \right) e_{k-1}$$

$$= u_{k-1} + K_p \left(\frac{T}{2T_n} + 1 \right) e_k + K_p \left(\frac{T}{2T_n} - 1 \right) e_{k-1}$$



Implementierung

s-Übertragungsfunktionen		Regelalgorithmen	
Elementare Übertragungsglieder / Regelalgorithmen			
P	$G_R(s) = K_P$	$u_k = d_0 \cdot e_k$	$d_0 = K_P$
I	$G_R(s) = \frac{1}{T_I \cdot s}$	$u_k = d_0 \cdot e_k + d_1 \cdot e_{k-1} + c_1 \cdot u_{k-1}$	$d_0 = \frac{T/2}{T_I}; d_1 = \frac{T/2}{T_I}; c_1 = 1$
D - T ₁	$G_R(s) = \frac{T_D \cdot s}{1 + T_d \cdot s}$		$d_0 = \frac{T_D}{T_d + T/2}; d_1 = -\frac{T_D}{T_d + T/2}; c_1 = \frac{T_d - T/2}{T_d + T/2}$
Regelalgorithmen			
PI	$G_R(s) = K_P \cdot \frac{1 + T_n \cdot s}{T_n \cdot s}$	$u_k = d_0 \cdot e_k + d_1 \cdot e_{k-1} + c_1 \cdot u_{k-1}$	$d_0 = K_P \cdot \frac{T_n + T/2}{T_n}; d_1 = -K_P \cdot \frac{T_n - T/2}{T_n}; c_1 = 1$
PD - T ₁	$G_R(s) = K_P \cdot \frac{1 + T_{vp} \cdot s}{1 + T_d \cdot s}$		$d_0 = K_P \cdot \frac{T_{vp} + T/2}{T_d + T/2}; d_1 = -K_P \cdot \frac{T_{vp} - T/2}{T_d + T/2}; c_1 = \frac{T_d - T/2}{T_d + T/2}$
PID - T ₁	$G_R(s) = K_{PP} \cdot \frac{1 + T_{np} \cdot s}{T_{np} \cdot s} \cdot \frac{1 + T_{vp} \cdot s}{1 + T_d \cdot s}$	$u_k = d_0 \cdot e_k + d_1 \cdot e_{k-1} + d_2 \cdot e_{k-2} + c_1 \cdot u_{k-1} + c_2 \cdot u_{k-2}$ $1 = c_1 + c_2$	$d_0 = K_{PP} \cdot \frac{T_{np} + T/2}{T_{np}} \cdot \frac{T_{vp} + T/2}{T_d + T/2};$ $d_1 = -2 \cdot K_{PP} \cdot \frac{T_{np} \cdot T_{vp} - (T/2)^2}{T_{np} \cdot (T_d + T/2)}; c_1 = \frac{2 \cdot T_d}{T_d + T/2}$ $d_2 = K_{PP} \cdot \frac{T_{np} - T/2}{T_{np}} \cdot \frac{T_{vp} - T/2}{T_d + T/2}; c_2 = -\frac{T_d - T/2}{T_d + T/2}$

Implementierung

Implementierung der Regelalgorithmen

1. Bei der Implementierung der Regelalgorithmen wird die sogenannte Summenform bevorzugt, bei der anstelle der geschlossenen Differenzengleichung

$$u_k = d_0 \cdot e_k + d_1 \cdot e_{k-1} + c_1 \cdot u_{k-1}$$

eine (beispielsweise beim PI-Regler) klare Aufteilung in Proportional- und Integralanteil erfolgt (siehe Tabelle unten):

$$u_k = K_p \cdot e_k + u_{I,k}$$

$$u_{I,k} = u_{I,k-1} + d_I \cdot (e_k + e_{k-1})$$

$$\text{mit } d_I = \frac{d_0 + d_1}{2}, \quad K_p = \frac{d_0 - d_1}{2} \quad (\text{Voraussetzung } c_1 = 1, \text{ wie PI-Regler}).$$

Die Darstellung ist mathematisch im Ergebnis gleichwertig, vermeidet aber Differenzen relativ großer Zahlen, die insbesondere bei Prozessoren mit Festpunktarithmetik, geringen Bitbreiten und großen Abtastfrequenzen (T klein) zu großen numerischen Ungenauigkeiten führen können.

Weitere Vorteile der Summenform:

- Bessere Interpretierbarkeit von Werten
- Einfachere Implementierung einer ARW-Maßnahme

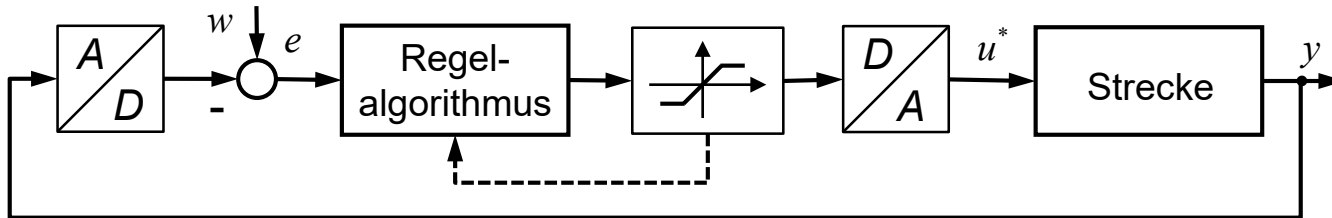
Implementierung

Regelalgorithmen - Summenform			
PI	$G_R(s) = K_P \cdot \left(1 + \frac{1}{T_n \cdot s}\right)$	$u_k = K_P \cdot e_k + u_{I,k}$ $u_{I,k} = u_{I,k-1} + d_I \cdot (e_k + e_{k-1})$	$d_I = K_P \cdot \frac{T}{2T_n}$
PD $- T_1$	$G_R(s) = K_P \cdot \left(1 + \frac{T_v \cdot s}{1 + T_d \cdot s}\right)$	$u_k = K_P \cdot e_k + u_{D,k}$ $u_{D,k} = c_D \cdot u_{D,k-1} + d_D \cdot (e_k + e_{k-1})$	$d_D = K_P \cdot \frac{T_v}{T_d + \frac{T}{2}}; c_D = \frac{T_d - T/2}{T_d + T/2}$
PID $- T_1$	$G_R(s) = K_P \cdot \left(1 + \frac{1}{T_n \cdot s} \cdot \frac{T_v \cdot s}{1 + T_d \cdot s}\right)$	$u_k = K_P \cdot e_k + u_{I,k} + u_{D,k}$ $u_{I,k} = u_{I,k-1} + d_I \cdot (e_k + e_{k-1})$ $u_{D,k} = c_D \cdot u_{D,k-1} + d_D \cdot (e_k - e_{k-1})$	$d_I = K_P \cdot \frac{T/2}{T_n}; d_D = K_P \cdot \frac{T_v}{T_d + T/2}$ $c_D = K_P \cdot \frac{T_d - T/2}{T_d + T/2}$

Implementierung

2. ARW-Maßnahme

Die Anti-Reset-Windup-Maßnahme (ARW – s. Kap. 4) soll verhindern, dass der I-Anteil eines Regler im Fall der Stellgrößenbegrenzung (der Regelkreis verhält sich wie offen) unkontrolliert hochläuft und somit zu einem unerwünschten Einschwingverhalten nach Verlassen der Stellgrößenbegrenzung führt:



Bei digitalen Reglern in Summenform kann eine solche ARW-Maßnahme sehr einfach durch Speicherung entsprechend angepasster Vergangenheitswerte implementiert werden:

```

e_k  = w - y;           % Regelabweichung
uI_k = uI_k_1 + dI * (e_k + e_k_1); % Integrator
u_k  = Kp * e_k + uI_k;  % Summe des Regelalgorithmus
if u_k >= umax { u_k = umax; } % Begrenzung
e_k_1 = e_k;            % Speichern Regelabweichung
uI_k_1 = u_k - Kp * e_k; % Speichern Integral

```

Implementierung

3. Task des Regelalgorithmus

Der periodisch mit fester Abtastschrittweite T auszuführende Regelalgorithmus wird auf einem μP oder μC üblicherweise als ein durch einen Timer-Interrupt ausgelöstes Prozess (Timer-Interrupt-Task) implementiert:

```

/* ---- definitions and global variables ----- */
#define T 1.0e-4                                /* 100 us  step size */
...

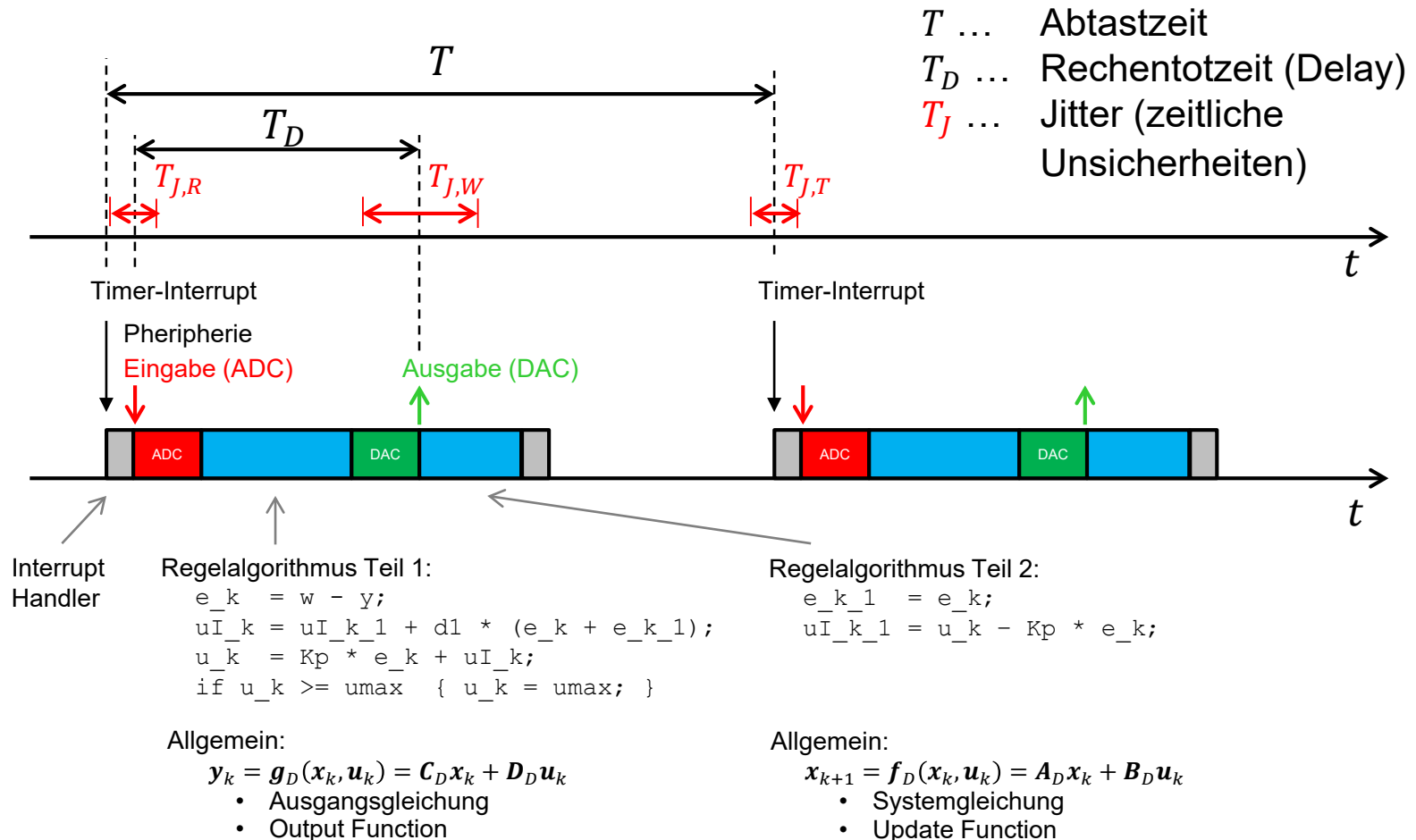
/* ---- sample rate isr ----- */
void timerA_interrupt(void)
{
    y = ADC_read(Ch_y);                          /* read control variable by ADC */
    e_k = w - y;                                  /* deviation */
    uI_k = uI_k_1 + dI * (e_k + e_k_1);           /* integration */
    u_k = Kp * e_k + uI_k;                        /* control algorithm*/
    if u_k >= umax { u_k = umax; }                /* limitation */
    DAC_write(Ch_u, u_k);                        /* write actuating variable by DAC */
    e_k_1 = e_k;                                  /* store deviation */
    uI_k_1 = u_k - Kp * e_k;                      /* store integral */
}

/* ---- main ----- */
void main(void)
{
    RTLIB_SRT_START(T, timerA_interrupt);        /* start sample rate timer */
    ...
}

```

Implementierung

4. Task-Timing bei Regelalgorithmen:



Implementierung

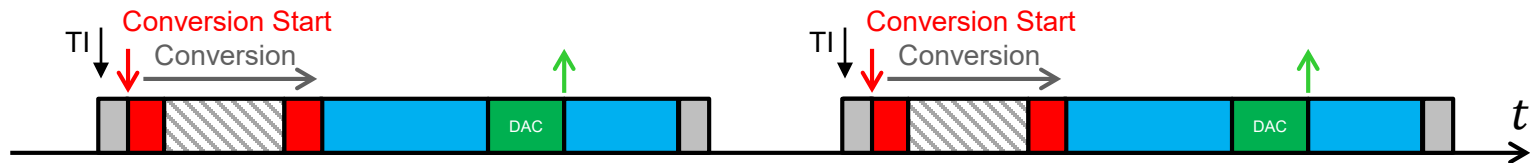
Anmerkungen:

- Die Details der Implementierung hängen von der verwendeten Plattform ab: μC , μP , FPGA, mit oder ohne Echtzeitbetriebssystem, ...
- Die Rechentotzeit T_D kann (muss) beim Reglerentwurf berücksichtigt werden.
- Jitter kann nur schwer berücksichtigt werden (Stochastische Methoden) und führt definitiv immer zu einem schlechteren Regelungsverhalten.
- Bei preiswerten (langsamen) ADC (oder solchen die z. B. in μC integriert sind) ist die Umsetzzeit T_C (conversion time) teilweise relativ groß im Vergleich zur Abtastschrittweite T : $T_C \geq 20\% \cdot T$.
- Eine digitale Vorverarbeitung (überabtastender ADC mit digitalem Anti-Aliasing-Filter) ermöglicht einen weitgehend verzögerungsfreien Zugriff auf die Peripherie.
- Ein genaue Analyse des Timing-Verhaltens ist ohne unterstützende Software schwierig
→ Zugriff auf interne Größen; Zeitmessung.
- Trick: Messung der Reaktionszeit mit Signalgenerator und Oszilloskop über die Peripherie (ggf. mit Dummy-Code).

Implementierung

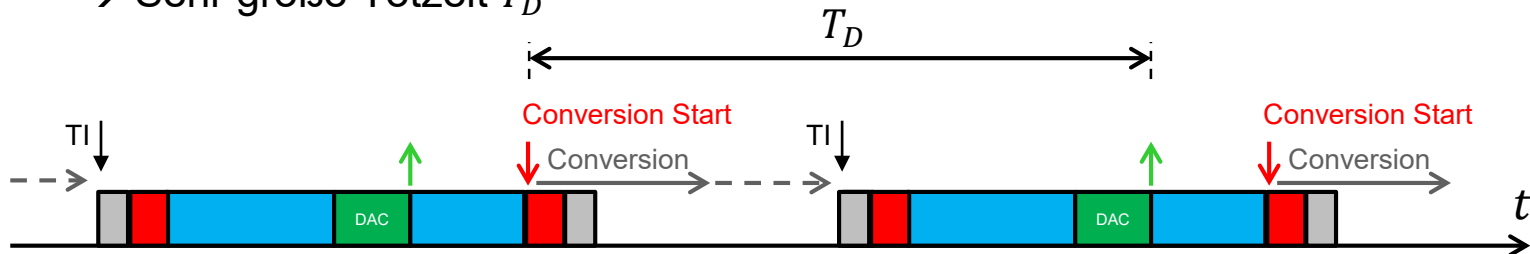
5. Anstoßen der ADC (langsame ADC ohne digitale Signalvorverarbeitung)

I. Prozess wartet (Polling Loop) → CPU ist blockiert für andere Prozesse

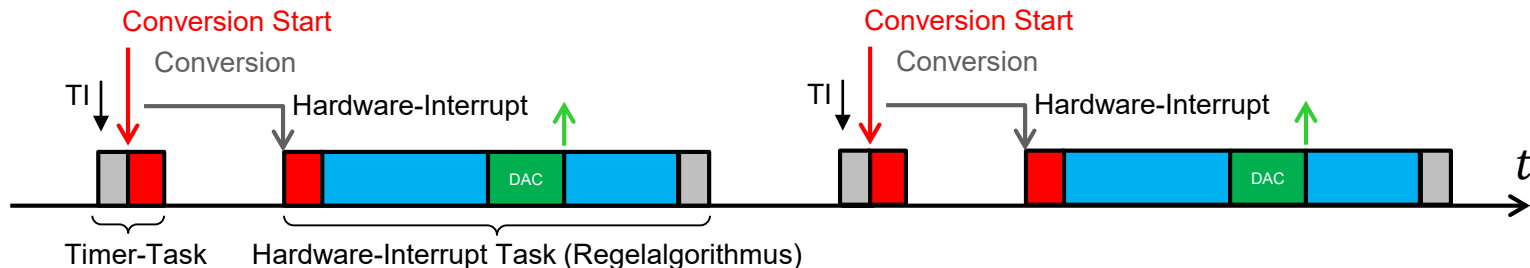


II. Anstoßende ADC-Conversion am Ende des Regelalgorithmus

→ Sehr große Totzeit T_D



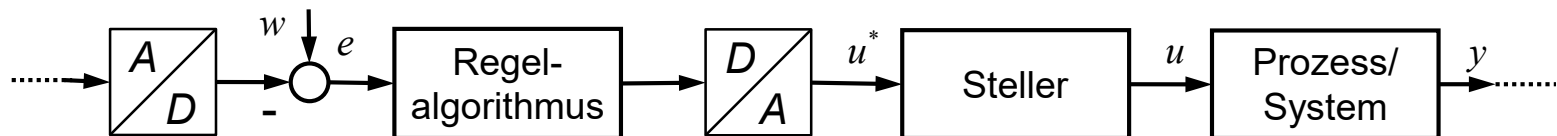
III. Interrupt durch ADC → separate Task



Implementierung

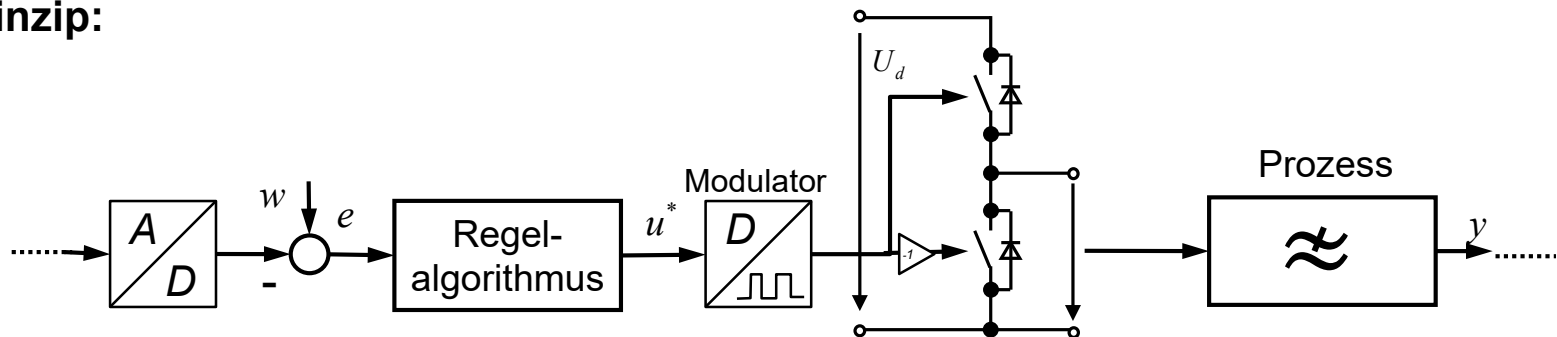
6. Stellgröße und Stellglied

Seltene Implementierungsvariante (bzw. Modellvorstellung)

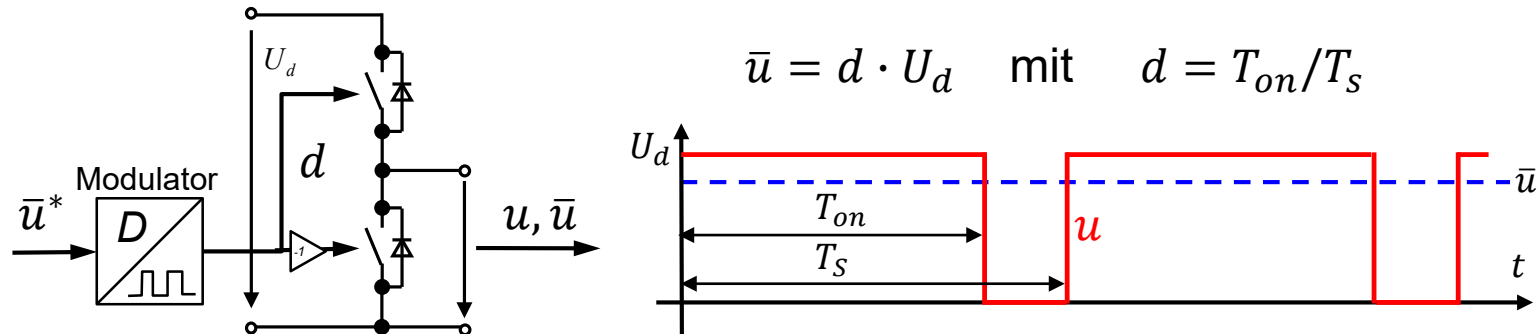


Die (aller)meisten Stellglieder weisen eine leistungselektronische (schaltende) Endstufe auf, die z. B. einen elektromechanischen Wandler (z. B. Elektromotor, hydraulisches/pneumatisches Ventil) oder einen elektrischen oder thermischen Prozesse (Schaltnetzteil, elektrische Heizung) steuert.

Prinzip:



Implementierung



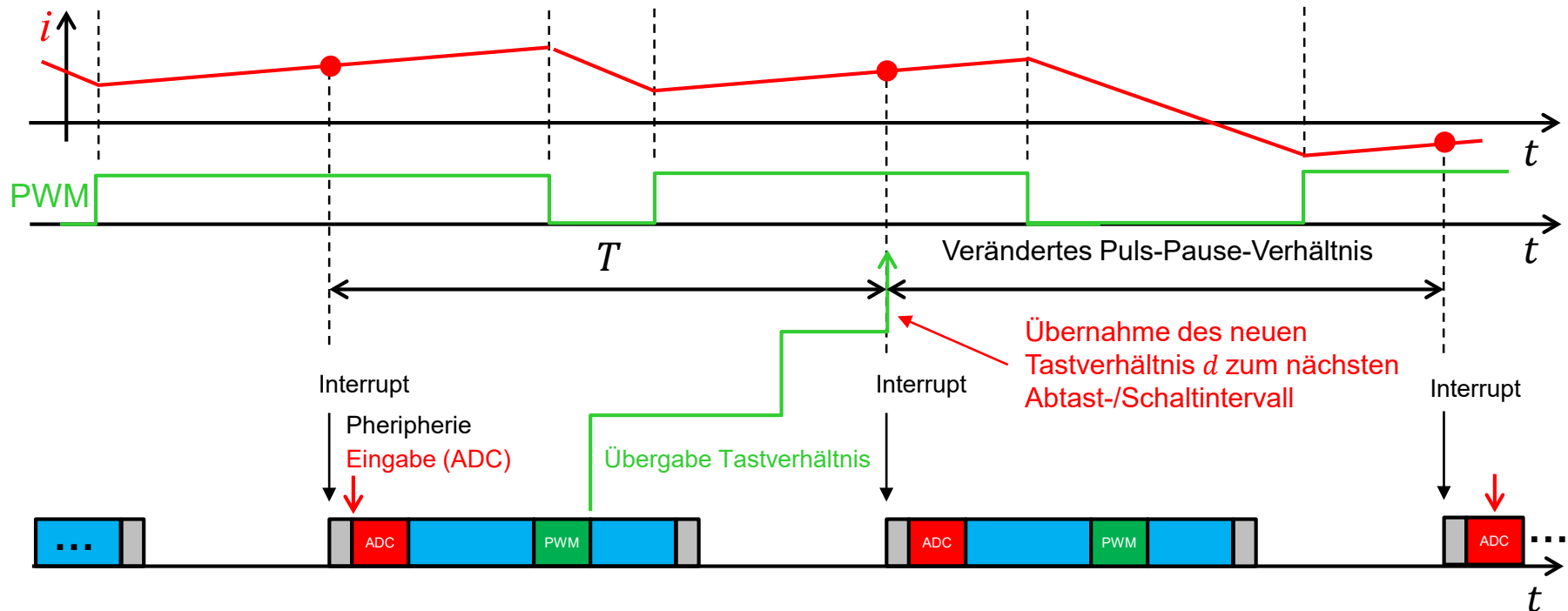
Modulator: Z. B. programmierbarer digitaler Zähler

Schaltperiodendauer T_s und Abtastfrequenz T

- $T_s \ll T$: Asynchroner Betrieb
- $n \cdot T_s = T$, $n = 1, \dots, 10$: Synchronisation zwischen digitaler Regelung und Modulation
- $T_s = T$: Typischer Betrieb (z. B. Antriebe)
- $T_s/2 = T$: Sonderfall (hohe Dynamik)
- $T_s/2 > T$: Nicht sinnvoll

Implementierung

Synchronisation: Pulsmittensynchrone Abtastung



$T_s = T$ mit Synchronisation über den Modulator auf die Pulsmitte (ausgelöst z. B. über Hardware-Interrupt). Der gemessene Strom entspricht ohne Filterung dem Mittelwert. Alternative Synchronisation (z.B. Pulsanfang) möglich aber unüblich.

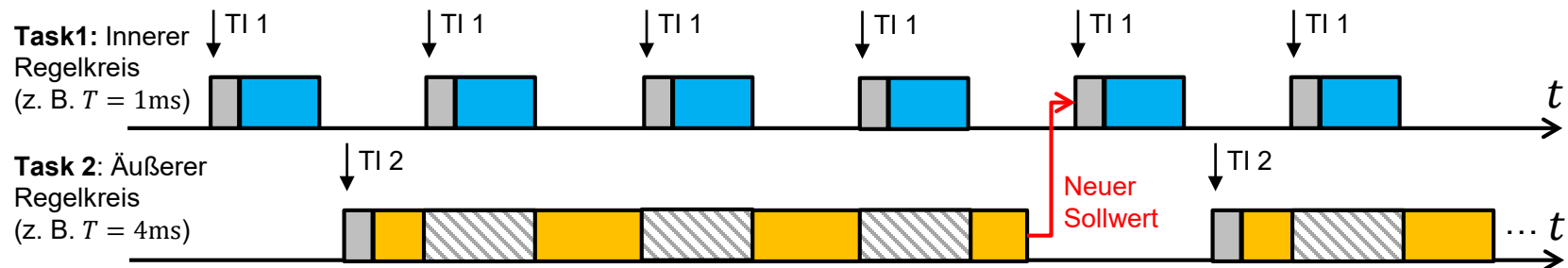
Gesamtzeit: $T_{tot} = T_{d(\text{Rechen})} + T_{\text{Stromrichter}} + T_{\text{AH}} = \left(1 + \frac{1}{4} + \frac{1}{2}\right) T_s = 1,75 \cdot T_s$

Implementierung

7. Mehrere Regelaufgaben:

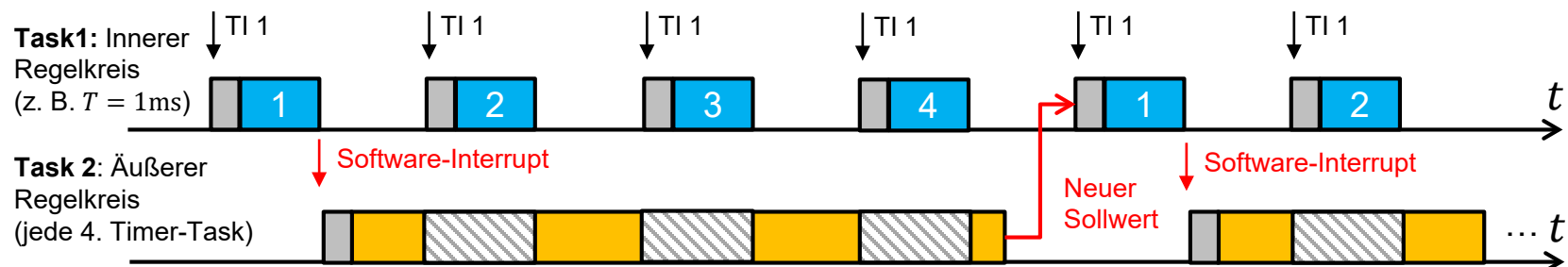
(z. B. Kaskadenregelung mit unterschiedlichen Abtastschrittweiten)

I. Asynchron (zwei Timer-Interrupts):



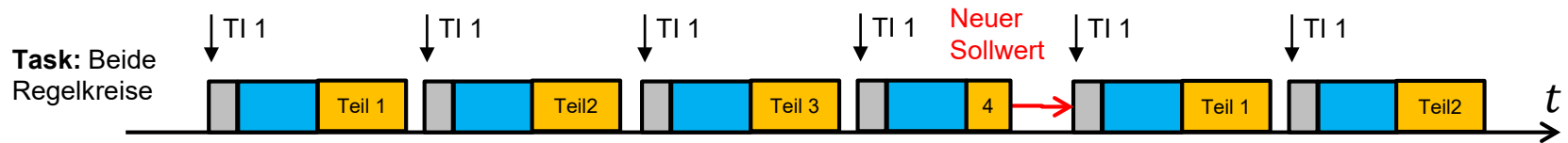
- Timer und Tasks sind nicht synchronisiert → keine deterministische Totzeit beim äußeren Regler → ggf. Schwebungseffekte (Aliasing)

II. Synchron (durch Software-Interrupt):



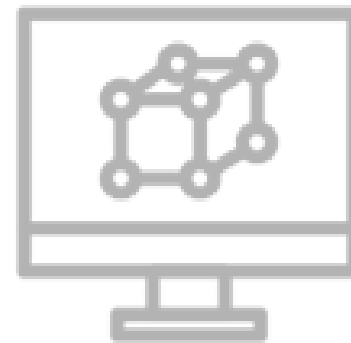
Implementierung

III. Synchron (eine Task):



- Einfacheres Task-Handling
- Günstige (zeitliche) Aufteilung erforderlich

Simulation



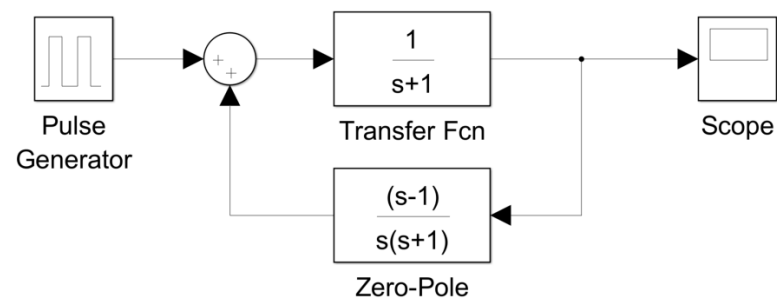
Simulation

Bedeutung der Simulation in der RT, Mechatronik und anderen Bereiche:

- Vorhersage des Systemverhaltens (z. B. auch Wetterprognosen, ...)
- Analysen (z. B. Potentialanalysen, Nachstellen von Unfallszenarien, ...)
- Validierung von Systementwürfen (z. B. Reglerentwurf, Gesamtentwurf, ...)
- Erprobung von Steuerungen und Regelungen vor der Inbetriebnahme
- Systemoptimierung (modellbasierte Ermittlung von Gütekriterien)
- Echtzeitsimulation (z. B. Hardware-in-the-Loop-Simulation, Predictive Maintenance, Zustandsschätzung, Überwachung,...)
- ...

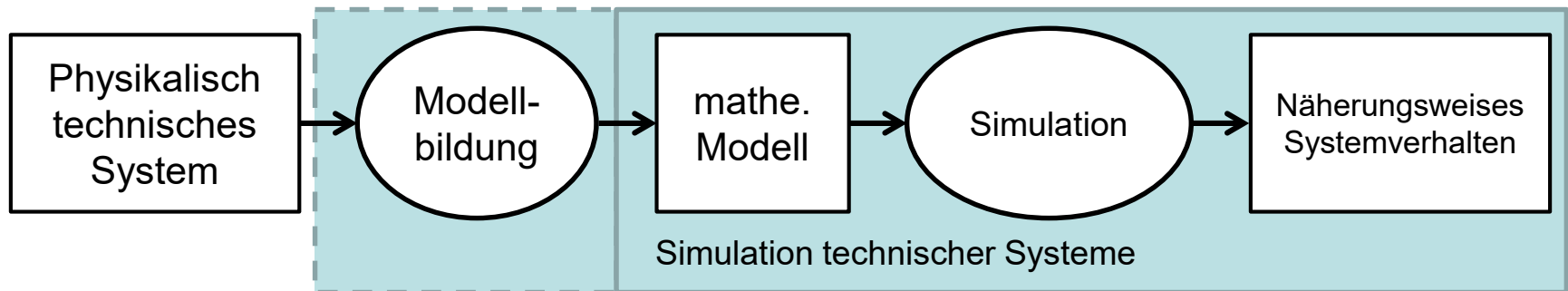
Fragen:

- Wie arbeiten Simulationswerkzeuge (wie z. B. Simulink)?
 - Numerik: Wie lösen sie die zugrundeliegenden Anfangswertprobleme?
- Was ist ggf. zu beachten?



Simulation

Einführung in die digitale Simulation



Wichtig: Die Modellbildung beeinflusst das Modell und damit die Art der Simulation
→ Es ist keine vollkommen isolierte Betrachtung der Simulation möglich.

Unterscheidung:

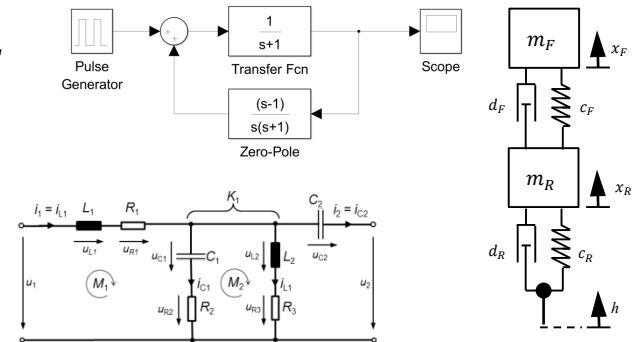
- *Simulations-Werkzeuge:* Das Modell liegt (zumindest als Blockdiagramm) vor und wird über die implementierten Algorithmen simuliert (Beispiel: Simulink ohne Zusatzbibliotheken wie z. B. SimPowerSystems)
- *Modellierungswerkzeuge:* Erzeugung eines mathematischen Modells aus einer z. B. topologischen oder geometrischen Darstellung heraus (Beispiel Pspice, Simplorer, Maxwell3D, usw.) - i. d. R. gekoppelt mit dem Simulationswerkzeug.

Simulation

Systeme mit konzentrierten und verteilten Parametern:

„Konzentrierte“ Prozesse → gewöhnliche DGL:

- Elektrische Schaltungen
- Mechanische Mehrkörpersysteme
- Hydraulik, Pneumatik

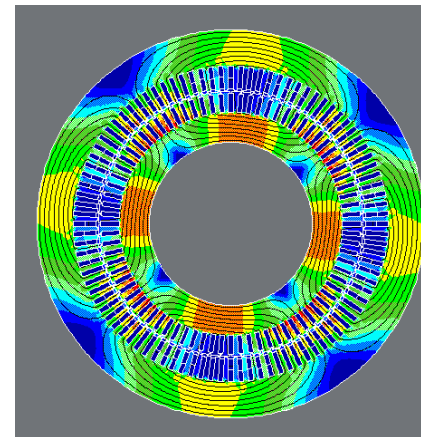


Simulation: Klassische Integrationsverfahren

„Verteilte“ Prozesse → partielle DGL:

- Strömungsprozesse
- Elektromagnetische Felder
- Kontinuumsmechanik

Simulation: Finite Elemente Methoden (FEM)



Feldbildung im
Asynchronmotor

Simulation

Simulation von Systemen mit konzentrierten Parametern:

→ Lsg. gewöhnlicher DGL / Anfangswertprobleme

Systeme, die einer gewöhnlichen DGL gehorchen, lassen sich meist als ZRM schreiben:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$$

$$\mathbf{y} = \mathbf{g}(\mathbf{x}, \mathbf{u}, t)$$

nichtlinear zeitvariant (allg.)

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$$

$$\mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u}$$

LTI

Hinweis: Alle Simulink-Blöcke sind in dieser Form implementiert,
auch benutzerdefinierte Systeme (s. Simulink s-Functions)

Ziel der digitalen Simulation:

Ermittlung (Approximation) von zeitdiskreten Werten $\mathbf{x}(t_i)$ des Zustandsgrößenvektors und $\mathbf{y}(t_i)$ der Ausgangsgrößen zu den Zeitpunkten $t_1, t_2, t_3 \dots t_n$ bei einem gegebenen Startwert $\mathbf{x}(t_0)$ (→ Anfangswertproblem) mittels eines Digitalrechner.

Schreibweise: $\mathbf{x}(t_k) = \mathbf{x}_k$ und $T_k = t_{k+1} - t_k \dots$ Abtastzeit

Hinweis: Bei konstanter Abtastzeit $T_k = T \quad \forall k$ gilt: $t_k = kT$ und $\mathbf{x}_k = \mathbf{x}(kT)$

Simulation

Die Berechnung der Ausgangsgleichung ist bei bekannten Zustandsgrößen unproblematisch:

$$\mathbf{y}(t_k) = \mathbf{g}(\mathbf{x}(t_k), \mathbf{u}(t_k), t_k) \quad \text{oder} \quad \mathbf{y}_k = \mathbf{g}(\mathbf{x}_k, \mathbf{u}_k, t_k)$$

Für die Lösung der Zustandsgrößen mit $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$ muss eine Lösung gefunden werden. Dies führt praktisch immer auf eine Differenzengleichung der Form

$$\mathbf{x}(t_{k+1}) = \mathbf{f}_D(\mathbf{x}(t_k), \mathbf{x}(t_{k-1}), \dots, \mathbf{x}(t_{k-n}), \mathbf{u}(t_k), \dots, \mathbf{u}(t_{k-m}))$$

$$\text{oder} \quad \mathbf{x}_{k+1} = \mathbf{f}_D(\mathbf{x}_k, \mathbf{x}_{k-1}, \dots, \mathbf{x}_{k-n}, \mathbf{u}_k, \mathbf{u}_{k-1}, \dots, \mathbf{u}_{k-m})$$

bzw. ein äquivalent darstellbares Differenzengleichungssystem. $\mathbf{f}_D$ ist dabei eine allgemeine Funktion in der Differenzengleichung.

Simulation

Klassische Integrationsverfahren:

In der Praxis wird für die Implementierung von Simulations-Tools in der Regel auf klassische Integrationsverfahren zurückgegriffen, da diese gegenüber anderen Methoden (*Filtermethode, augmentierte Systeme*) eine gute Algorithmisierbarkeit und Verallgemeinerbarkeit aufweisen.

Idee: Integration der Systemgleichung

$$\dot{x}(t) = f(x(t), u(t), t) \quad \text{mit Startwert} \quad x(t_k) = x_k$$

über ein geeignetes Intervall t_k bis t_{k+1} :

$$x(t_{k+1}) = x(t_k) + \int_{t_k}^{t_{k+1}} f(x(\tau), u(\tau), \tau) d\tau$$

Problem: Die exakte Lösung ist wegen $x(\tau)$ im Integral u. beliebigem $u(t)$ ausgeschlossen.

Lösungsansatz: Approximation des Integral $\rightarrow$ numerische Integration:

$$x(t_{k+1}) = x(t_k) + \int_{t_k}^{t_{k+1}} f d\tau \approx x(t_k) + I(x_k, x_{k-1}, \dots, x_{k-n}, u_k, u_{k-1}, \dots, u_{k-n})$$

(Die Verfahren unterscheiden sich durch die Integralapproximation.)

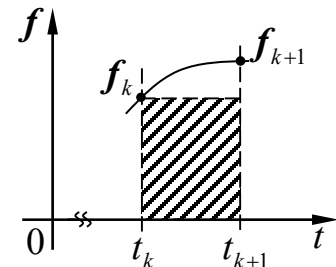
Simulation

Klassische numerische Integrationsverfahren:

Euler-Verfahren (Vorwärts-Euler, expliziter Euler, klassisches Eulerv.)

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + T \mathbf{f}(\mathbf{x}(t_k), \mathbf{u}(t_k), t_k) \quad \text{mit} \quad T = t_{k+1} - t_k$$

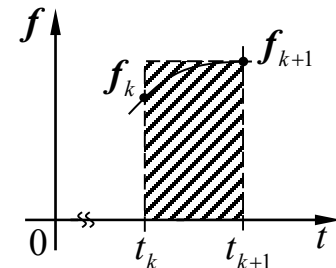
geschrieben $\mathbf{x}_{k+1} = \mathbf{x}_k + T \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k)$



Rückwärts-Euler (impliziter Euler, BDF oder First Difference)

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + T \mathbf{f}(\mathbf{x}(t_{k+1}), \mathbf{u}(t_{k+1}), t_{k+1}) \quad \text{mit} \quad T = t_{k+1} - t_k$$

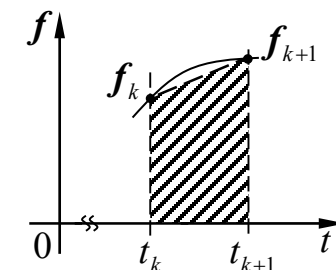
geschrieben $\mathbf{x}_{k+1} = \mathbf{x}_k + T \mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1})$



Trapezregel (Tustin)

$$\mathbf{x}(t_{k+1}) = \mathbf{x}(t_k) + \frac{T}{2} (\mathbf{f}(\mathbf{x}(t_{k+1}), \mathbf{u}(t_{k+1}), t_{k+1}) + \mathbf{f}(\mathbf{x}(t_k), \mathbf{u}(t_k), t_k))$$

geschrieben $\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{2} (\mathbf{f}(\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, t_{k+1}) + \mathbf{f}(\mathbf{x}_k, \mathbf{u}_k, t_k))$



Simulation

Allgemeine Form:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{I}(\mathbf{x}_{k+1}, \mathbf{x}_k, \mathbf{x}_{k-1}, \dots, \mathbf{x}_{k-n}, \mathbf{u}_{k+1}, \mathbf{u}_k, \dots, \mathbf{u}_{k-n}, t_{k+1}, t_k, \dots, t_{k-n})$$

Verfahrensklassen:

1. Implizite / explizit Verfahren:

Explizite Verfahren: $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{I}(\mathbf{x}_k, \dots)$ die rechte Seite enthält $\mathbf{x}_{k+1}$ nicht.

- Explizite Verfahren lassen eine direkte Berechnung in jedem Rechenschritt/Abtastschritt zu.

Implizite Verfahren: $\mathbf{x}_{k+1} = \mathbf{x}_k + \mathbf{I}(\mathbf{x}_{k+1}, \mathbf{x}_k, \dots)$ die rechte Seite enthält $\mathbf{x}_{k+1}$.

- Für implizite Verfahren müssen spezielle Methoden angewendet werden (s. u.).

Hinweis: Das Euler-Verfahren (Vorwärts-Euler) ist ein *explizites Verfahren*, während der *Rückwärts-Euler* und die *Trapez-Regel implizit* sind (s. o.).

Simulation

2. Einschritt- / Mehrschrittverfahren:

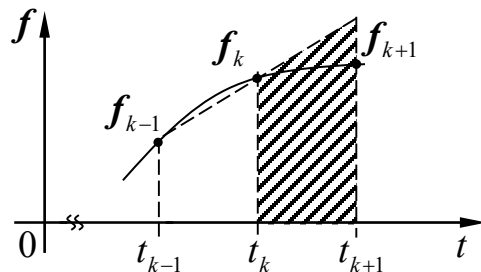
Numerische Integrationsverfahren bei denen die Integralnäherung nur über einen Abtastschritt hinweg erfolgen werden als **Einschrittverfahren** bezeichnet. Die Differenzengleichung enthält keine „älteren“ Vergangenheitswerte:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + I(\mathbf{x}_{k+1}, \mathbf{x}_k, \mathbf{u}_{k+1}, \mathbf{u}_k)$$

Numerische Integrationsverfahren bei denen die Integralnäherung mehrere Abtastschritte (zeitdiskrete Stützstellen) nutzt werden als **Mehrschrittverfahren** bezeichnet. Dies bedeutet, dass die Differenzengleichung auch „ältere“ Vergangenheitswerte enthält:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + I(\mathbf{x}_{k+1}, \mathbf{x}_k, \mathbf{x}_{k-1}, \dots, \mathbf{x}_{k-n}, \dots)$$

Bsp.: Adams-Bashforth-2:



$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{T}{2} (3\mathbf{f}(\mathbf{x}_k, \mathbf{u}_k) - \mathbf{f}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}))$$

Simulation

Numerische Stabilität:

Eine der wichtigsten Eigenschaften eines numerischen Integrationsverfahrens ist seine **numerische Stabilität**, d. h. dass die Zustandsvariablen bei begrenzter, konstanter Eingangsgröße gegen einen Endwerte streben (konvergieren).

Hinweis: Die numerische Stabilität eines Integrations-/Simulationsverfahrens ist von der Stabilität des simulierten Systems zu unterscheiden:
→ Stabile Systems können in der Simulation instabil werden (und umgekehrt)!

Überprüfung der numerische Stabilität:

Für LTI Systeme kann das einfache Testsystem

$$\dot{x} = \lambda \cdot x \quad \text{mit} \quad \lambda \in \mathbb{C} \quad \dots \text{Eigenwert}$$

verwendet werden (System 1. Ordnung ohne Eingangsgröße mit komplexem Eigenwert).

Hinweise:

- Es kann gezeigt werden, dass der Test auch äquivalente Aussagen für LTI-Systeme n -ter Ordnung (mit n Eigenwerten) liefert.
- Für nichtlineare Systeme ist eine allgemeine Aussage nicht möglich. Meist kann aber über eine Linearisierung im Arbeitspunkt eine lokale Stabilitätsaussage getroffen werden.

Simulation

Stabilität des expliziten Euler-Verfahrens:

Wird $\dot{x} = \lambda \cdot x$ mit $\lambda \in \mathbb{C}$ in das explizite Eulerverfahren eingesetzt, erhält man als Rekursions-/Verfahrensvorschrift:

$$x_{k+1} = x_k + T f(x_k, u_k, t_k) = x_k + \lambda T \cdot x_k = (1 + \lambda T)x_k = a(\lambda T) \cdot x_k$$

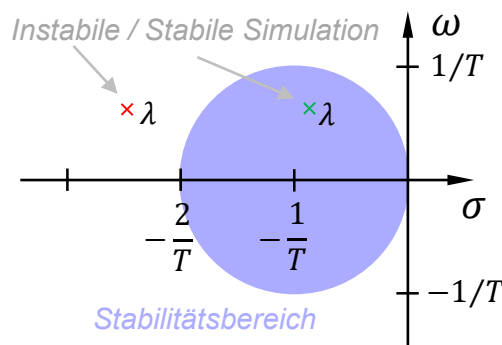
Der bei konstantem λT konstante Faktor $a(\lambda T) \in \mathbb{C}$ bestimmt mit

$$x_{k+1} = a(\lambda T) \cdot x_k$$

die Entwicklung von x_k und wird daher als *Stabilitätsfunktion* bezeichnet. Die entstehende Folge (x_k) von Zustandswerten konvergiert (hier gegen 0) nur dann, wenn gilt:

$$|a| = |a(\lambda T)| = |1 + \lambda T| < 1.$$

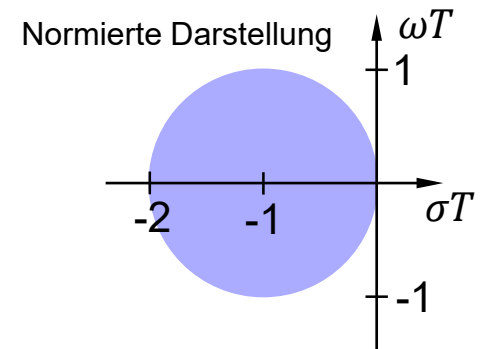
Dies lässt sich mit $\lambda = \sigma + j\omega$ als *Stabilitätsbereich* in der komplexen Ebene markieren:



Eulerverfahren:

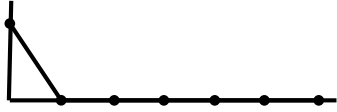
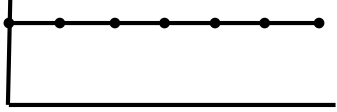
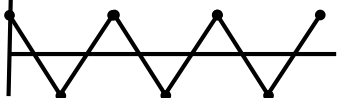
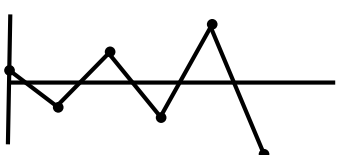
Kreis des Radius $1/T$ um $-1/T$

Hinweis: Der Stabilitätsbereich kann durch Verkleinerung von T (Abtastzeit) vergrößert werden.



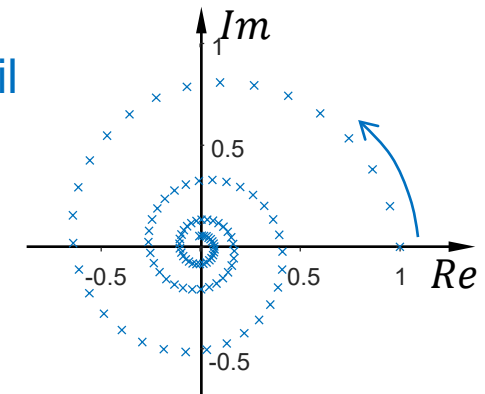
Simulation

Reelles $\lambda \rightarrow a \in \mathbb{R}$:

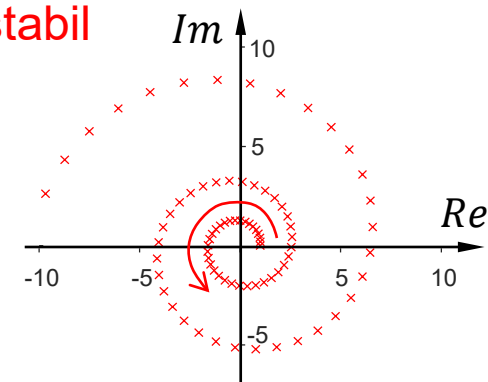
$0 < a < 1$	
$a = 0$	
$-1 < a < 0$	
$a = 1$	
$a = -1$	
$a > 1$	
$a < -1$	

Komplexes $\lambda \rightarrow a \in \mathbb{C}$:

Stabil

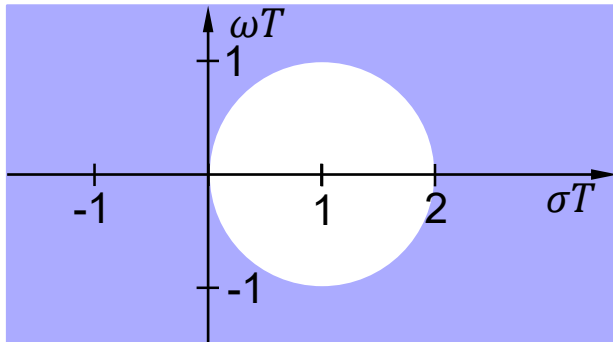


Instabil



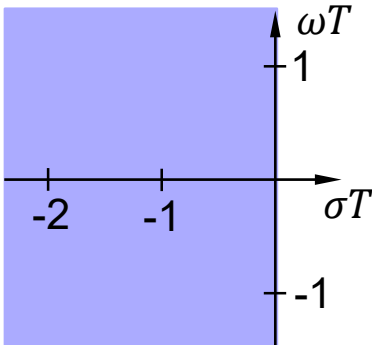
Simulation

Rückwärts-Euler-Verfahren:



Stabilitätsbereich: Komplette komplexe Ebene, außer dem Einheitskreis um +1.

Stabilität der Trapezregel:



Stabilitätsbereich:
Linke Hälfte der komplexen Ebene.

© Regelungstechnik und Simulation – Prof. Dr.-Ing. T. Schulte

Sehr großer Stabilitätsbereich, aber viele naturgemäß instabile Systeme werden fälschlicherweise stabil simuliert. Im Vergleich zum Vorwärts-Euler-Verfahren ist das Rückwärts-Euler-Verfahren nicht genauer. Das Rückwärts-Euler-Verfahren ist wegen seines sehr großen Stabilitätsbereichs gerade in der *Echtzeitsimulation* sehr beliebt.

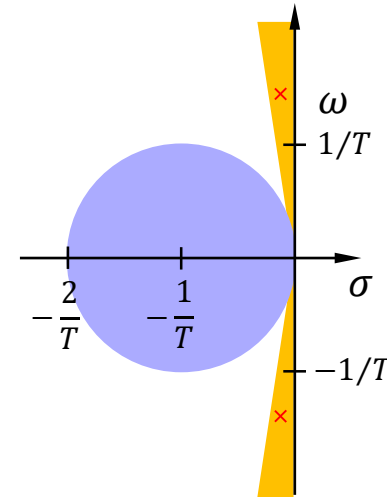
Stabile Systeme werden stabil simuliert und instabile werden instabil simuliert werden. Das Trapezregel-Verfahren ist somit bzgl. des Stabilitätsbereichs das ideale Verfahren.

Beide Verfahren heißen „*Absolut Stabil*“ (A-Stabilität).

Simulation

Hinweise:

- Vorwärts-Euler-Verfahren:
Probleme bereitet das Euler-Verfahren vor allem bei schwach gedämpften schwingungsfähigen Systemen (z. B. in der Mechanik – siehe Pole), da sich die „**Lücken**“ zwischen Stabilitätsbereich und imaginärer Achse bei Reduzierung der Schrittweite nur langsam schließen.
- Integrationsverfahren höher Ordnung (s. u.) haben zwar einen kleineren Stabilitätsbereich gegenüber der Trapezregel, sind aber dennoch genauer. Viele Integrationsverfahren sind so angelegt, dass ihr Stabilitätsbereich möglichst weite Teile der linken s-Halbebene einschließt und sich an die imaginäre Achse anschmiegen ohne in die rechte s-Halbebene zu ragen.
- Der Stabilitätsbereiche der impliziten Verfahren (Rückwärts-Euler, Trapezregel) wird durch die für die Auflösung der impliziten Form notwendigen Maßnahmen (z. B. zusätzliche numerische Schritte) je nach Umsetzung beeinträchtigt, d.h. i.d.R. reduziert.



Simulation

Fehler und Ordnung

Verfahrensfehler oder kurz „Fehler“, auch numerischer Fehler o. globaler Fehler:
 Akkumulierte Abweichung zwischen der exakten Lösung und dem Verfahren zum Zeitpunkt τ unter der Annahme, dass mit exakten Werten gestartet wurde:

$$e = e_g(\tau) = e_{g,k}^{(T)}(\tau) = |x(t_k) - x_k| = |x(\tau) - x_k| \quad \text{mit} \quad x(0) = x_0 \quad \text{und} \quad \tau = k \cdot T$$

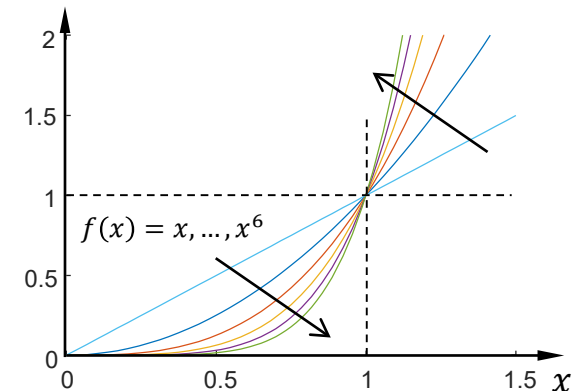
Ordnung / Fehlerordnung:

Die Ordnung eines Verfahrens p besagt, dass der globale Fehler e bei veränderlichem T maximal mit T^p anwächst, also gilt:

$$e \leq c \cdot T^p \quad \text{mit einer Konstanten } c.$$

Hinweise:

- Die Definition beinhaltet keine absolute Aussage über e , da c im Allgemeinen unbekannt ist sowie vom Verfahren und System abhängt.
- Hohe Fehlerordnungen kennzeichnen hochwertige Verfahren, da e bei Reduzierung von T schnell kleiner wird (s. rechts).



Simulation

Prädiktor-Korrektor-Verfahren:

Für das x_{k+1} auf der „rechten“ Seite der impliziten Iterationsvorschrift wird näherungsweise ein Wert durch ein anderes (explizites) Verfahren berechnet:

Prädiktorschritt: $x_{k+1}^P = x_k + T f(x_k, u_k)$ (expliziter Euler)

Korrektorschritt: $x_{k+1} = x_k + \frac{T}{2} \left(f(x_{k+1}^P, u_{k+1}) + f(x_k, u_k) \right)$ (Trapezregel)

Die Bezeichnung als „Prädiktor-Korrektor-Verfahren“ rührt offensichtlich von der zweistufigen Berechnung mit einem *prädizierenden* ersten Schritt und einem *korrigierenden* zweiten Schritt her.

Hinweis: Das oben dargestellte Verfahren wird meist als **Euler-Heun-Verfahren**, teilweise aber auch als **modifiziertes Euler-Verfahren** bezeichnet.

Simulation

Quadraturformel: (Vorschrift zur numerischen Berechnung eines Integrals)

$$\int_a^b f(\tau) d\tau \approx (t_{k+1} - t_k) \sum_i c_i f(t_i)$$

$f(t_i)$ Stützstellen mit $t_k < t_i < t_{k+1}$, c_i geeignet gewählte Koeffizienten.

Newton-Cotes-Formeln:

Basierend auf interpolierendes Polynom (Grad p) mit $p + 1$ äquidistanten Stützstellen.

Grad	Bezeichnung	Koeffizienten	
0	Rechteckregel	$c_i = 1$	
1	Trapezregel	$c_i = \frac{1}{2}; \frac{1}{2}$	
2	Simpson-Regel	$c_i = \frac{1}{6}; \frac{4}{6}; \frac{1}{6}$	
3	Pulcherrima-Regel	$c_i = \frac{1}{8}; \frac{3}{8}; \frac{3}{8}; \frac{1}{8}$	

Simulation

Beispiele für Verfahren höherer Ordnung:

Euler-Heun: Trapezregel als Quadraturformel mit Euler-Verfahren als Prädiktor:

$$(2\text{-stufig, Ord.1}) \quad \begin{aligned} x_{k+1}^P &= x_k + T f(x_k, u_k) \\ x_{k+1} &= x_k + \frac{T}{2} \left(f(x_{k+1}^P, u_{k+1}) + f(x_k, u_k) \right) \end{aligned}$$

Heun-Verfahren: Abwandlung des Euler-Heun (andere Stützstelle)

$$(2\text{-stufig, Ord.2}) \quad \begin{aligned} x^P &= x_k + \frac{2}{3} T f(x_k, u_k) && (\text{expliziter Euler auf } \frac{2}{3}\text{-Intervall}) \\ x_{k+1} &= x_k + \frac{T}{4} \left(3f\left(x^P, u_{k+\frac{2}{3}}\right) + f(x_k, u_k) \right) \end{aligned}$$

Runge-Kutta 3.Ordnung: Basierend auf Simpson-Regel:

$$(3\text{-stufig, Ord. 3}) \quad \begin{aligned} x^{P1} &= x_k + \frac{1}{2} T f(x_k, u_k) && (\text{expliziter Euler auf Intervallmitte}) \\ x^{P2} &= x_k - T f(x_k, u_k) + 2T f\left(x^{P1}, u_{k+\frac{1}{2}}\right) && (\text{Intervallende}) \\ x_{k+1} &= x_k + \frac{T}{6} \left(f(x_k, u_k) + 4f\left(x^{P1}, u_{k+\frac{1}{2}}\right) + f(x^{P2}, u_{k+1}) \right) \end{aligned}$$

Simulation

Runge-Kutta 4.Ordnung: Basierend auf Simpson-Regel:

(4-stufig, Ord. 4) $x^{P1} = x_k + \frac{1}{2}Tf(x_k, u_k)$ (expliziter Euler auf Intervallmitte)

$x^{P2} = x_k + \frac{1}{2}Tf(x^{P1}, u_{k+\frac{1}{2}})$ (wiederholt Euler auf Intervallmitte)

$x^{P3} = x_k + Tf(x^{P2}, u_{k+\frac{1}{2}})$ (Euler Intervallende)

$x_{k+1} = x_k + \frac{T}{6} \left(f(x_k, u_k) + 2f(x^{P1}, u_{k+\frac{1}{2}}) + 2f(x^{P2}, u_{k+\frac{1}{2}}) + f(x^{P3}, u_{k+1}) \right)$

3/8-Regel (Kutta): Basierend auf Pulcherima-Regel:

(4-stufig, Ord. 4) $x^{P1} = x_k + \frac{1}{3}Tf(x_k, u_k)$ (expliziter Euler auf 1/3 Intervalllänge)

$x^{P2} = x_k - \frac{1}{3}Tf(x_k, u_k) + Tf(x^{P1}, u_{k+\frac{1}{3}})$

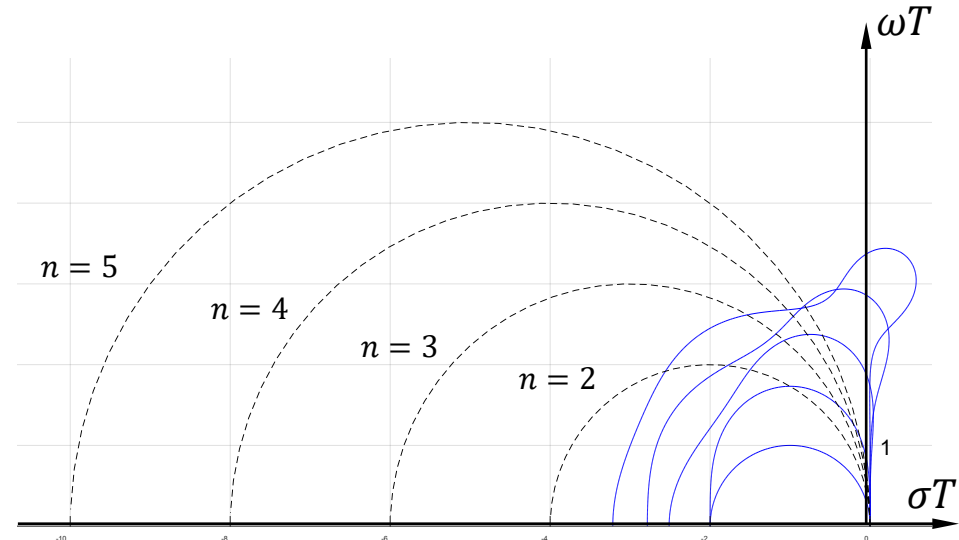
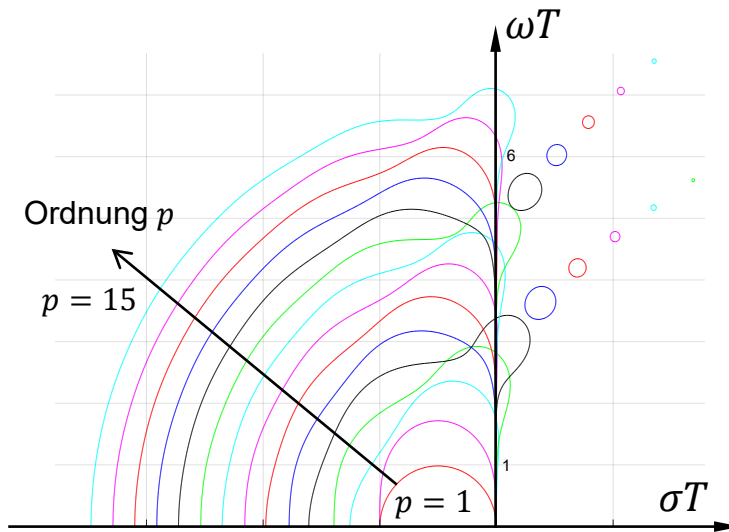
$x^{P3} = x_k + Tf(x_k, u_k) - Tf(x^{P1}, u_{k+\frac{1}{3}}) + Tf(x^{P2}, u_{k+\frac{2}{3}})$

$x_{k+1} = x_k + \frac{T}{8} \left(f(x_k, u_k) + 3f(x^{P1}, u_{k+\frac{1}{3}}) + 3f(x^{P2}, u_{k+\frac{2}{3}}) + f(x^{P3}, u_{k+1}) \right)$

Simulation

Stabilität:

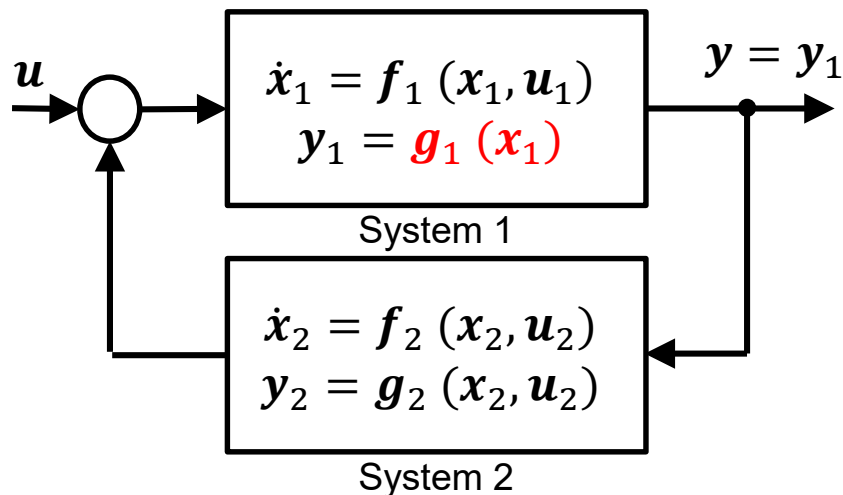
Mit wachsender Stufenzahl und Ordnung p nimmt der Bereich der numerischen Stabilität der Einschritt-Verfahren zu (wegen der immer vorliegenden Symmetrie ist nur der obere Bereich der komplexen Ebene dargestellt):



Die Halbkreise zeigen den erweiterten Stabilitätsbereich des Euler-Verfahrens bei n -facher Abtastfrequenz. Dies entspricht bzgl. des Rechenaufwands für eine identische Simulationszeit einem ($p = n$)-stufigen Verfahren. Zwar sind die Stabilitätsbereiche des Eulerverfahrens in der Fläche größer, aber die kritischen Bereichen nahe der imaginären Achse werden nicht erfasst.

Simulation

Systeme in Wirkungsplänen (z. B. Simulink-mdl)



Euler-Verfahren:

$$\begin{aligned}
 & \dots \\
 & y_{1,k} = g_1(x_{1,k}) \quad \leftarrow \text{Start} \\
 & u_{2,k} = y_{1,k} \\
 & y_{2,k} = g_2(x_{2,k}, u_{2,k}) \quad \text{Koppelgl.} \\
 & u_{1,k} = u + y_{2,k} \\
 & x_{1,k+1} = x_{1,k} + T f_1(x_{1,k}, u_{1,k}) \\
 & x_{2,k+1} = x_{2,k} + T f_2(x_{2,k}, u_{2,k}) \\
 & \dots
 \end{aligned}$$

Hinweis:

- Weisen alle Blöcke in einem geschlossenen Wirkungskreis (Schleife) eine direkte Abhängigkeit der Ausgangs- von der Eingangsgröße auf (sprungfähige Systeme, Direct-Feed-Through), entsteht eine algebraische Schleife (algebraic loop). Dies beinhaltet eine implizite, algebraische Beziehung, die durch gesonderte numerische Verfahren (z. B. Newton-Verfahren) aufgelöst werden muss.

Simulation

Schrittweitensteuerung:

- Bei Integrationsverfahren mit variabler Schrittweite wird die Schrittweite T in Abhängigkeit des lokalen Verfahrensfehlers automatisch gesteuert.
- Dadurch lässt sich die Simulation wegen der im Allgemeinen größeren Schrittweite T gegenüber Verfahren mit konstanter Schrittweite T beschleunigen, während gleichzeitig eine hinreichende Berücksichtigung von kritischen Stellen (wie z. B. Schaltvorgänge) durch lokale Reduzierung der Schrittweite erfolgt.

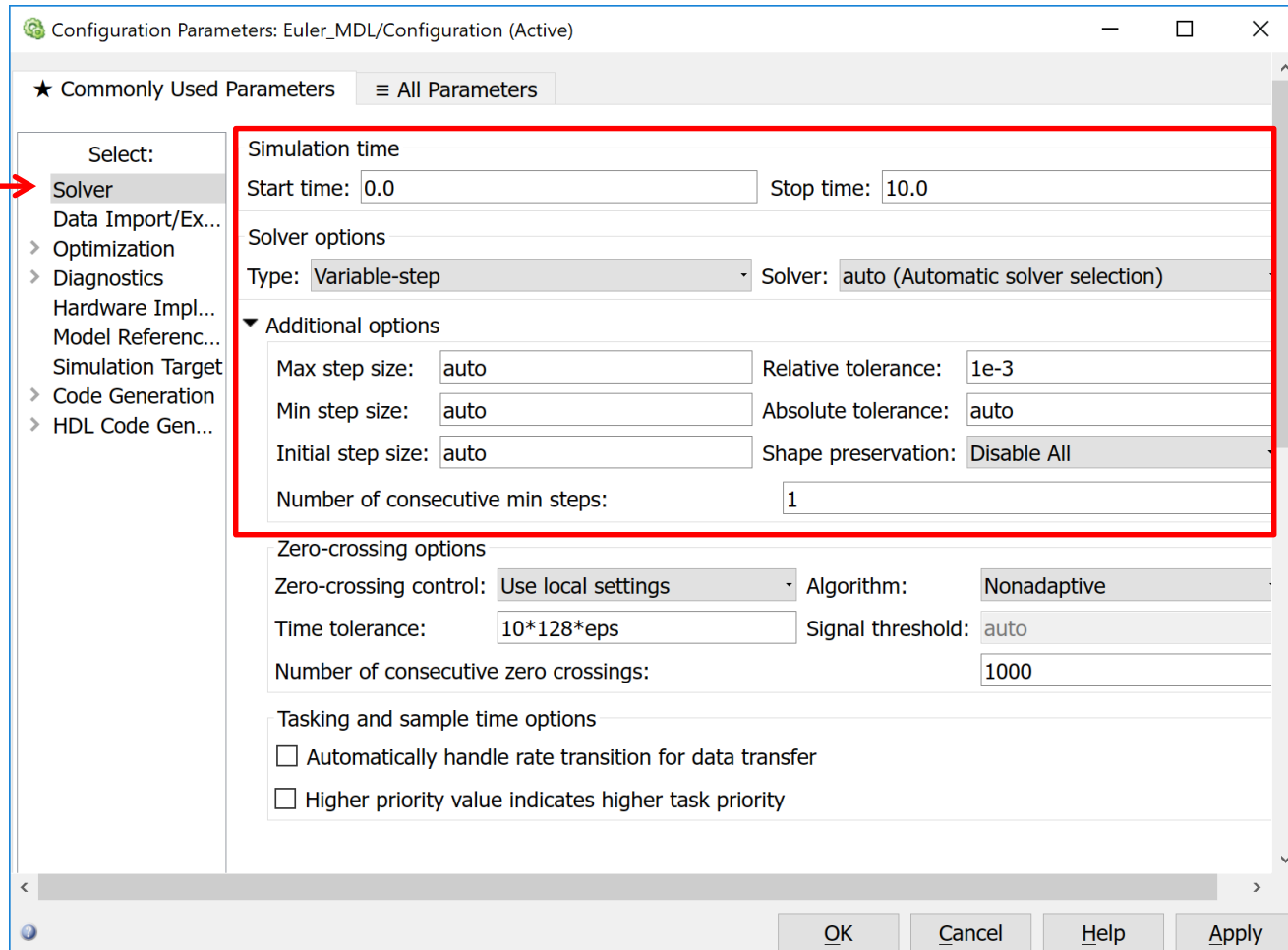
Zur Schrittweitensteuerung über den Verfahrensfehler existieren prinzipiell zwei Ansätze:

1. Parallelrechnung eines Integrationsverfahren mit zwei unterschiedlichen Schrittweiten und der darauf basierenden Schätzung des lokalen Fehlers.
2. Parallelrechnung mit unterschiedlicher Ordnung bzw. Stufen p und der darauf basierenden Schätzung des lokalen Fehlers.
Z. B. Matlab: ode45 → eingebettetes RK-Verfahren der Ordnung 4 und 5.

Im Allgemeinen wird auf das zweite Verfahren zurückgegriffen, das gegenüber der ersten Lösung einen reduzierten Rechenaufwand beinhaltet, da Zwischengrößen wiederverwendet werden können.

Simulation

Matlab/Simulink: Einstellen der Simulationsparameter



Simulation

Einstellungen für das Integrationsverfahren:

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Type	Variable-step	Integrationsverfahren mit variabler Schrittweite (default); für den RTW (Codegenerierung für den Echtzeitbetriebe) nicht wählbar.
	Fixed-step	Integrationsverfahren mit fester Schrittweite; Einstellung für den RTW.

Hinweis:

- Die Seite *Solver* passt sich je nach Einstellung (z. B. Variable-step / Fixed-step oder Wahl des Verfahrens) an. So sind immer nur die Einstellungen zu sehen, die auf das gewählte Verfahren Einfluss haben.
- Die Vielzahl implementierter Verfahren ist gerechtfertigt, da für jedes Simulationsproblem ein bestimmtes Verfahren am besten geeignet ist. Entscheidend sind dabei Simulationsgeschwindigkeit Genauigkeit, notwendige Stabilitätseigenschaften (s. Eigenwerte) und die Häufigkeit der Strukturwechsel.

Simulation

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Solver (Fixed-step)	Discrete (no continuous states)	kein Verfahren aktiv
	ode8 (Dormand-Prince)	-
	ode5 (Dormand-Prince)	-
	ode4 (Runge-Kutta)	-
	ode3 (Bogacki-Shampine)	(default)
	ode2 (Heun)	-
	ode1 (Euler)	-
	ode14x (extrapolation)	-

ode N → Explizite Einschrittverfahren der Ordnung N

ode14x (extrapolation) → Implizites Extrapolationsverfahren

Simulation

<i>Einstellung</i>	<i>Optionen</i>	<i>Beschreibung</i>
Solver (Variable-step)	Discrete (no continuous states)	kein Verfahren aktiv
	ode45 (Dormand-Prince)	(default)
	ode23 (Bogacki-Shampine)	-
	ode113 (Adams)	-
	ode15s (stiff/NDF)	-
	ode23s (stiff/Mod. Rosenbrock)	-
	ode23t (Mod. stiff/Trapezoidal)	-
	ode23tb (stiff/TR-BDF2)	-

ode45 bzw. **ode23** → Explizite eingebettete Verfahren 4 und 5 bzw. 2 und 3 Ordnung.

„s“ (→ stiff) markierte Verfahren die für **steife** Systeme geeignete sind.

Steif ist eine System, das eine große Eigenwertspreizung aufweist, d.h. sehr schnelle und sehr langsame Vorgänge vereint.

Anwendung



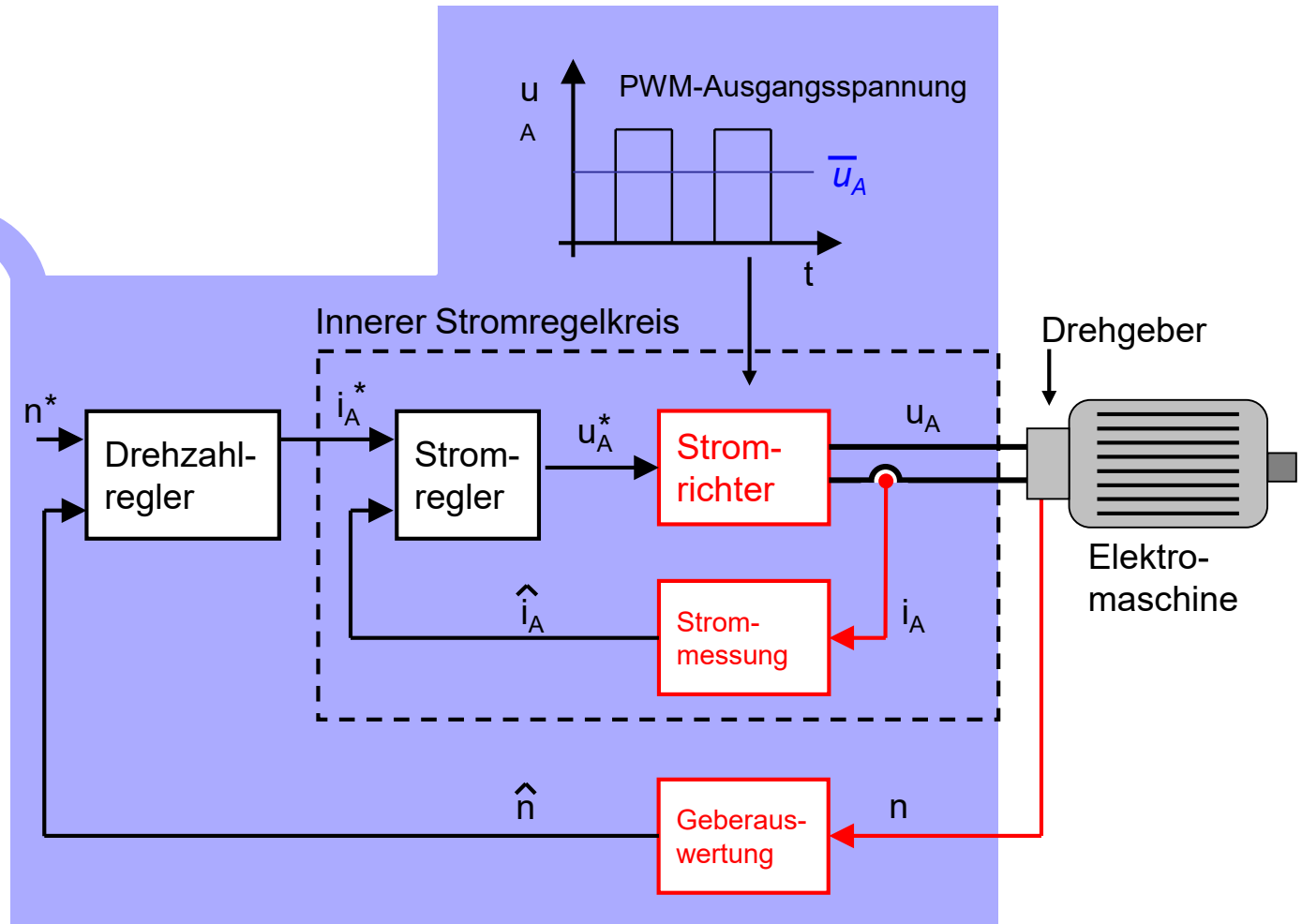
Anwendung

Antriebsregelung:

Servoregler



EM



Anwendung

Auslegung einer kaskadierten Strom-Drehzahlregelung

Annahme: Auslegung der Regler nach „Symmetrischem Optimum“ und Approximation als Verzögerungsglied 1.Ordnung (P-T1-Glied):

Stromregelung: $T_{ni} = 4T_{1i} = 8T_s$

T_{1i} ... kleinste Zeitkonstante im elektrischen System mit $1,5T_s$ bis $2T_s$ (hier $T_{1i} = 2T_s$)

T_s ... Schaltperiodendauer ($T_s = \frac{1}{f_s}$)

Drehzahlregelung: $T_{n\omega} = 4T_{1\omega} = 4T_{ni} = 32T_s$

$T_{1\omega}$... kleinste Zeitkonstante der Drehzahlregelung (ohne Messwertfilter), ist mit $T_{1\omega} = T_{ni}$ die Ersatzzeitkonstante der approx. Stromregelung

$T_{n\omega}$... Nachstellzeit und Ersatzzeitkonstante der approx. Geschwindigkeitsregelung

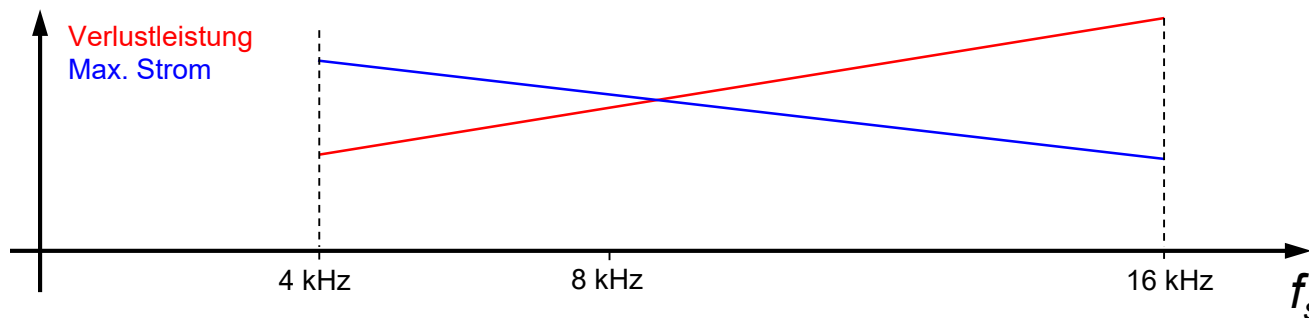
Fazit: Die Schaltperiodendauer $T_s = \frac{1}{f_s}$ bestimmt die kleinen Zeitkonstanten und damit insgesamt die Dynamik der überlagerten Regelkreise.

→ Wahl einer möglichst hohen Schaltfrequenz für hohe Regeldynamik.

Anwendung

Beispieldatenblatt eines Servoumrichters (anonymisiert):

Typ		A1	A2	A3
Bemessungsstrom	A	5	7	10
Kurzzeitgrenzstrom (2 s)	%	200	200	200
Bemessungsleistung	kW	3,5	5,5	7,0
Max. Strom $f_s = 4$ kHz	%	120	130	140
Max. Strom $f_s = 8$ kHz	%	100	100	100
Max. Strom $f_s = 16$ kHz	%	70	65	60

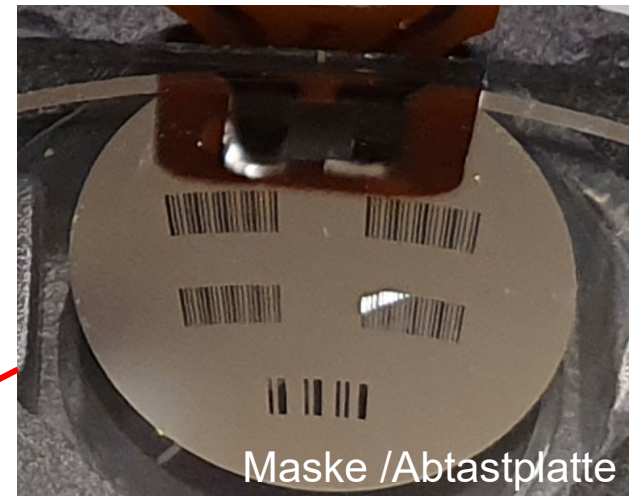
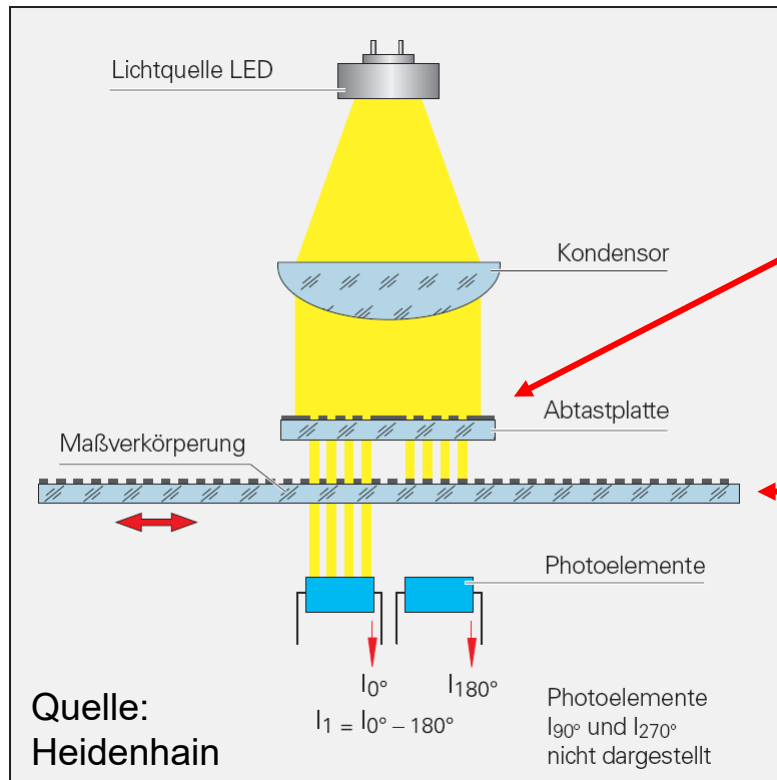


→ Dimensionierung als Kompromiss zwischen Effizienz, Kosten und Dynamik

Anwendung

Drehgeber:

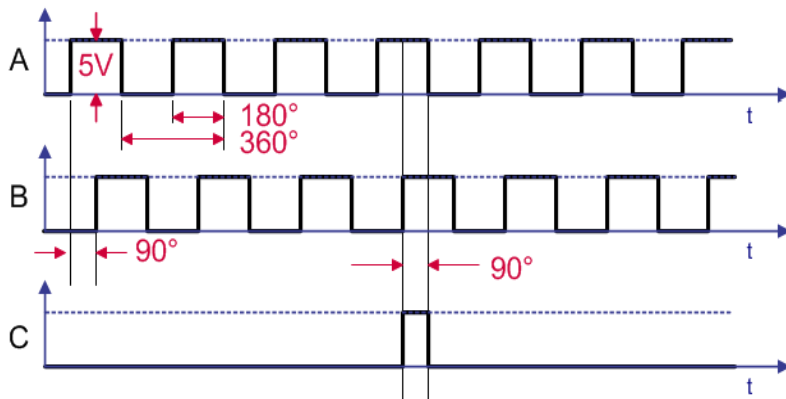
Inkrementalgeber, Sinusgeber, Absolutwertgeber sind häufig als optische Geber ausgeführt.



Anwendung

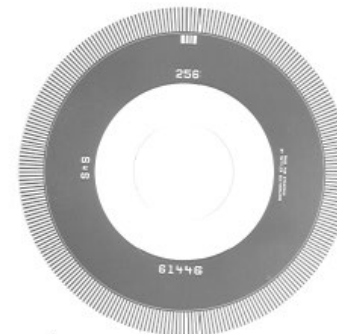
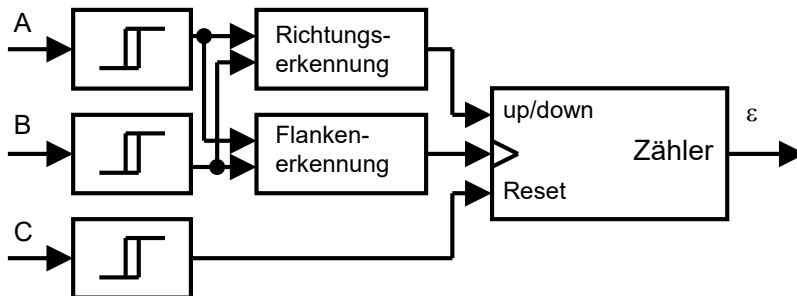
Inkrementalgebersignale und -auswertung

Die Codescheibe des Inkrementalgebers besteht aus Hell-/Dunkelfeldern (Bitzuständen 1/High und 0/Low). Durch Zählen der Inkremente wird die Position erfasst. Für die Drehrichtungserfassung dient eine zweite Code-Spur durch einen zweiten um 90° versetzten Abtaster. Mit einem zusätzlichen Nullimpuls (einmal pro mechanischer Umdrehung) wird die absolute Lage erfasst.



Digitale Ausgangssignale des Lagegebers

- Zwei Codespuren bzw. Abtaster mit 90° Versatz
- Zustandsautomat zur Richtungsauswertung
- Lage- bzw. Winkelerfassung
- Typisch: 1024, 2048 oder 4096 Striche / 360°
- Vierfachausswertung: vier Teilungen pro Puls/Strich werden unterschieden:
1024 Striche $\rightarrow$ 4096 Inkremente



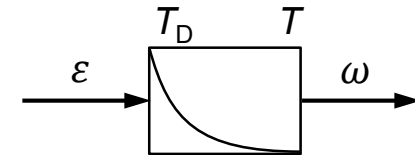
Anwendung

Drehzahl- und Lageerfassung:

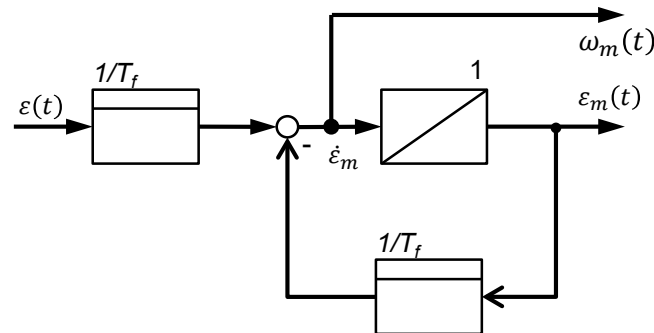
- Die meisten Drehgebersysteme sind Lagegeber.
- Die Drehzahl (Drehkreisfrequenz) wird durch differenzieren bzw. diskret durch Differenzbildung gewonnen:

➤ Idealer Differenzierer $\omega(t) = \frac{d\varepsilon(t)}{dt}$

➤ Realer Differenzierer: $G(s) = \frac{s}{1+s \cdot T}$ (D-T₁-Glieder)



- Kombinierter Filterung von Lage und Drehkreisfrequenz:



Anwendung

Auflösung der Drehzahl bei Impulszählungsverfahren:

Beispiel: Inkrementalgeber mit 2048 Strichen $\rightarrow$ 8192 Winkelsegment $\rightarrow$ Auflösung 0.0439°

Digitale Auswertung:

$$\varpi_R = \frac{\Delta \varepsilon}{T_{AB}} = \frac{2\pi}{T_{AB}} \frac{\Delta k}{k_G}$$

Δk ... überlaufene Teilungen

k_G ... Gesamtanzahl der Teilung

T_{AB} ... Abtastzeit; Typischer Wert
bei Drehzahlregelungen $T_{AB} = 200\mu\text{s}$

Quantisierung (kleinste messbare Drehfrequenz $\rightarrow \Delta k = 1$): $\Delta \varpi_R = \frac{2\pi}{T_{AB} \cdot k_G}$

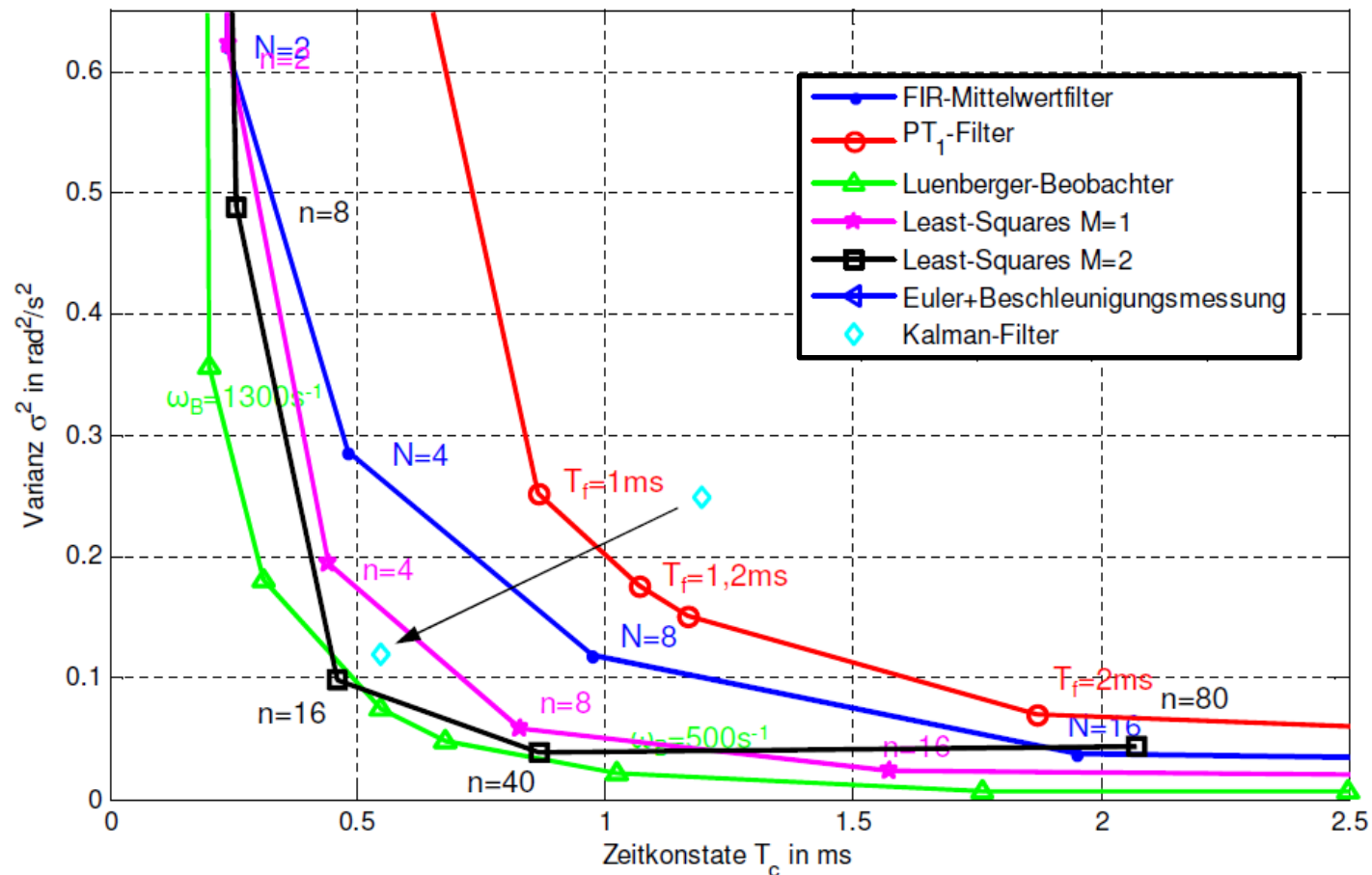
Im Beispiel: $k = 8192$ und $T_{AB} = 200\mu\text{s}$: $\Delta \varpi = 3,84 \text{ rad/s}$ oder $\Delta n = 36,6 \text{ min}^{-1}$

Verbesserungen:

- Filterung des Drehzahlmesswerts
- Erhöhung der Geberauflösung (z. B. Sinus-Geber)
- Drehzahlschätzverfahren (s. nächste Folie)
- Verbesserte Messverfahren: Überabtastung oder Beschleunigungssensor

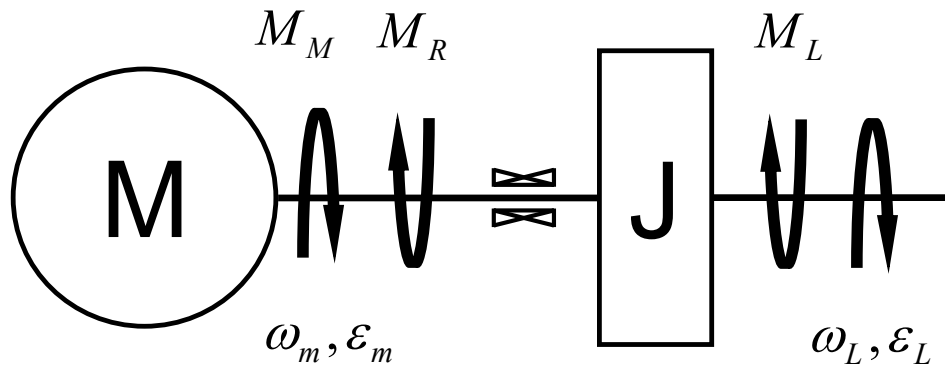
Anwendung

Verbesserung der Drehzahlerfassung (Verfahren nicht weiter erläutert)



Anwendung

Einmassensystem mit Reibung:

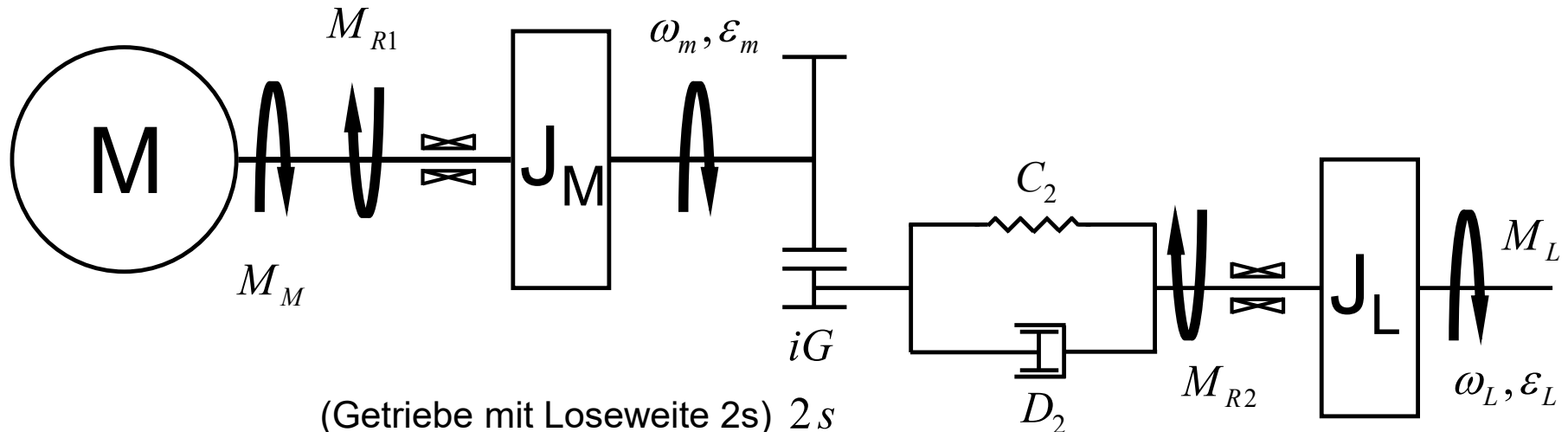


Starre Koppelung:

$$J = J_M + J_L$$

$$\omega = \omega_M = \omega_L$$

Zweimassensystem mit Getriebeispiel, Nachgiebigkeit und Reibung:



(Getriebe mit Loseweite 2s) 2s

Anwendung

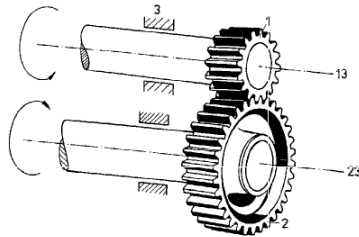
Gründe für eine nicht starre Koppelung:

- Kupplungen und Übertragungswellen:



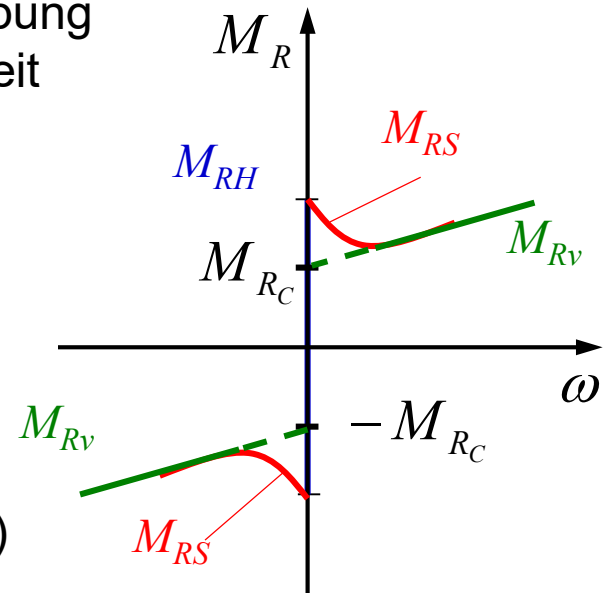
- Reduzierte Steifigkeit
- Lose (je nach Bauart)

- Getriebe:



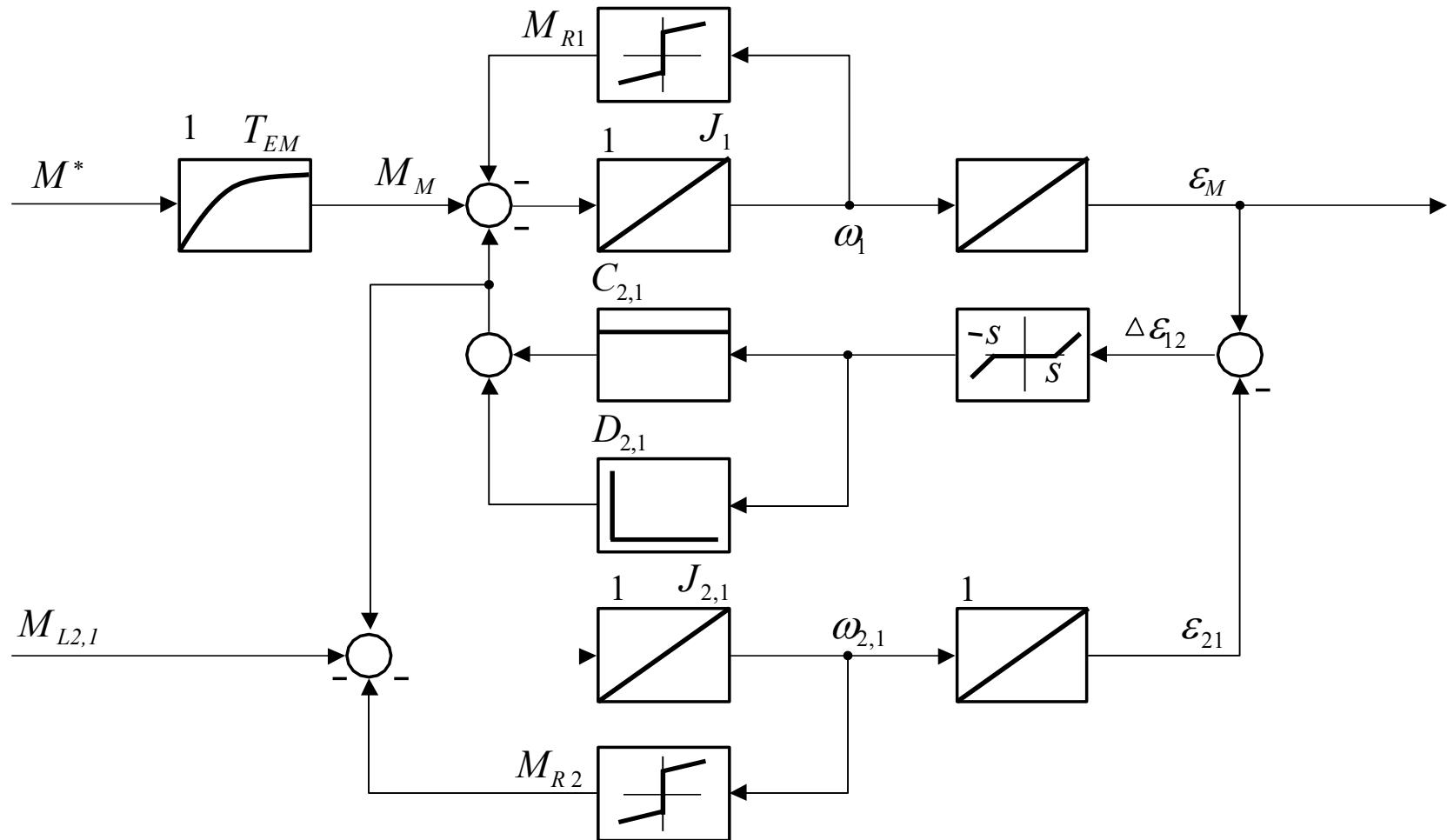
- Lose (je nach Bauart)
- Lastabhängige Reibung
- Reduzierte Steifigkeit

- Reibung:
 - M_{RC} - trockene Reibung, Coulombsche Reibung (konstant)
 - M_{Rv} fluidische Reibung (geschwindigkeitsabhängig)
 - M_{RH} Haftreibung
 - M_{RS} Stribeck-Reibung (Übergang von Haft- auf Gleitreibung)



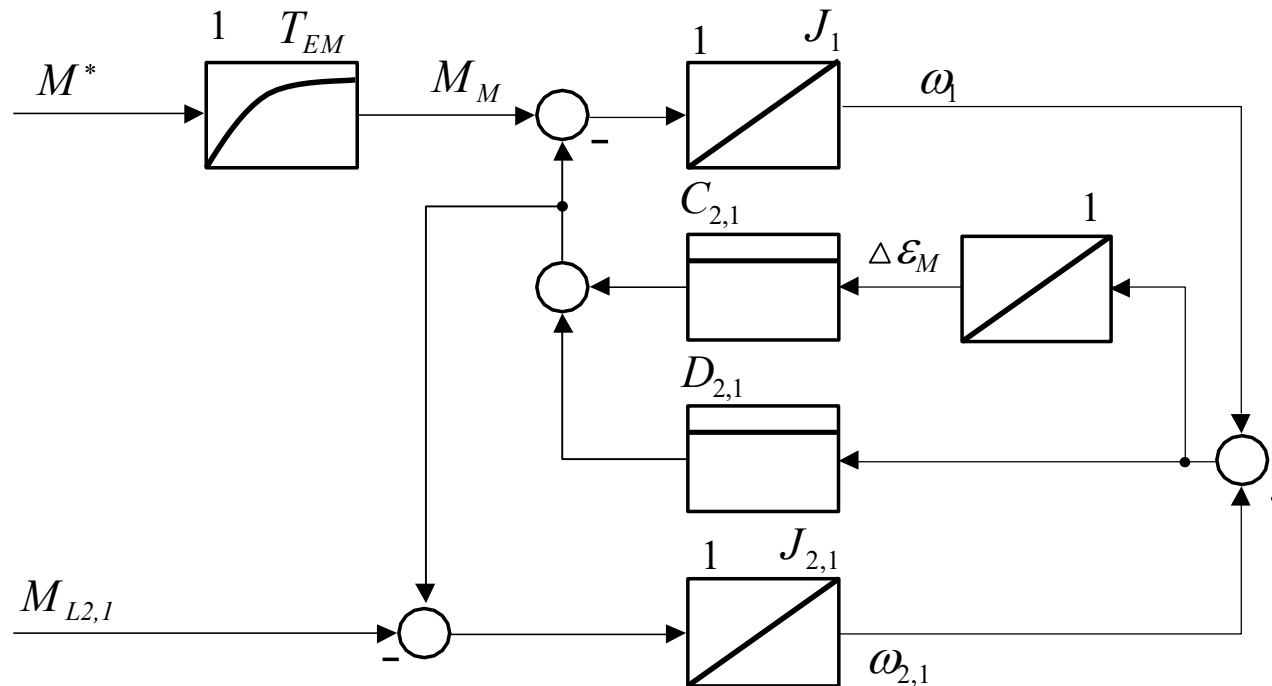
Anwendung

Wirkungsplan ZMS mit Getriebespiel, Nachgiebigkeit und Reibung:



Anwendung

Wirkungsplan ZMS mit Nachgiebigkeit aber ohne Getriebeispiel und Reibung:



Anwendung

Lineares Zweimassensystem:

Charakteristisch für das ZMS ist die Resonanz ω_R und die Antiresonanz ω_A . Sollte ω_A in für die Drehzahl-Lageregelung relevanten Frequenzbereich liegen, kann die aus dem Einmassenmodell entwickelte Regelung nicht und unverändert übernommen werden.

→ Z. B. Zustandsregler (RT2)

Alle realen Systeme sind Zwei- oder Mehrmassensystem. Bei einem steifen mechanische Aufbau liegen ω_A und ω_R in einem so hohen Frequenzbereich, dass als Näherung eine starre Koppelung angenommen werden kann.

Beispiel:

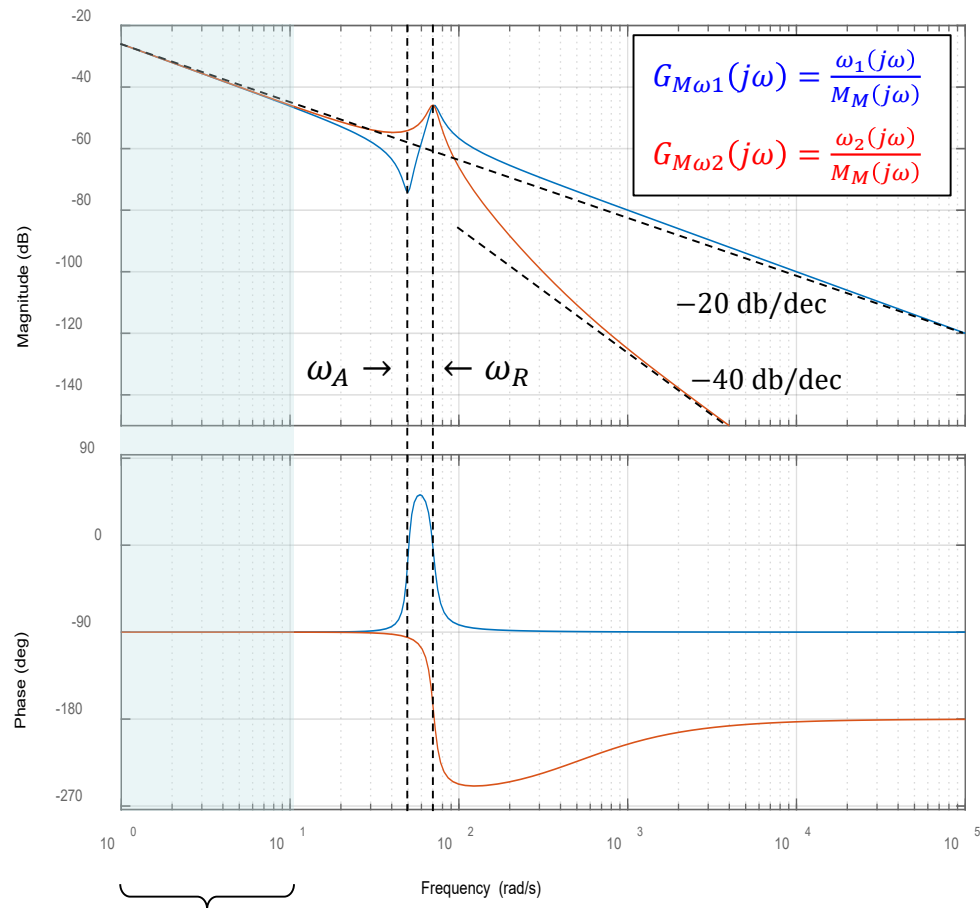
$$C = 1000 \text{ Nm/rad}$$

$$D = 50 \text{ Nms/rad}$$

$$J_m = 10 \text{ kg/m}^2$$

$$J_L = 300 \text{ kg/m}^2$$

$$i_g = 5$$



Anwendung

Zwei- oder Mehrmassensystem mit Getriebespiel, Nachgiebigkeit und Reibung stellen eine besondere regelungstechnische Herausforderung dar. Bei Regelungsabweichungsaufgaben mit besonderen Anforderungen an Dynamik und Präzision muss schon bei der Konstruktion des Systems auf diverse Dinge geachtet werden. Insbesondere **Lose und Haftreibung** sind regelungstechnisch nicht kompensierbar:

Vermeidung von Lose:

- Hochwertige Kupplungen, drehsteife Übertragungswellen
- Keine oder wenig Getriebestufen
(Direktantrieb → Verschlechterung des Massenverhältnisses)
- Spielfrei(-arme) vorgespannte Getriebe (Erhöhung der Reibung)

Vermeidung von Haft- oder Stribeck-Reibung:

- Keine oder wenige hochwertige Wellendichtungen
- Vermeidung von Gleitlagern
- Keine Kugel- oder Rollenlager mit Dichtelementen
- Magnetlagertechnik

*Vielen Dank,
eine erfolgreiche Prüfungsphase
und einen erholsamen Sommer!*